Live Ninja — Agentic Capability Expansion Review

Prepared: 2026-07-19 · **Repo:** JeremyProffittOrg/live-ninja · **Owner:** Jeremy Proffitt **Method:** 16-agent review workflow — 4 parallel subsystem readers (server/tools, clients, infra, product docs), then 6 themed ideation agents, then 6 adversarial feasibility verifiers that checked every suggestion against the actual code, IAM, contracts, and the repo cost posture. **84 suggestions total**, every one anchored to real files and honestly effort-rated (S under 1 day · M = 1-3 days · L = 1-2 weeks · XL over 2 weeks). "Verified with corrections" means the idea survives but the verifier fixed a wrong assumption — the correction is quoted inline.

Executive summary — your asks, answered

Your ask	Verdict	How
Start remote coding sessions ("ghost-cli", Claude/GitHub integration)	Fully buildable	GitHub tool family in the existing tool router: `@claude`-mention issue filing / `workflow_dispatch` of headless `claude -p`, `JOB#` tracking rows, webhook push-back, question relay, voice-gated merge. Batch/Fargate self-hosted runner for the Actions-free case.
Play videos / play media	Buildable (audio-first)	Android client-local media-key tools + the Google-Assistant search-and-play intent for native YouTube Music/Spotify playback over car Bluetooth; podcasts, article TTS, and generated briefing audio server-side.
News & updates ("from d." — truncated)	Buildable	Generic `fetch_feed` (RSS/Atom/JSON-feed) + GitHub digest + market prices, all recipe-pluggable. Clarify "d." and it becomes one more typed source.
A tool that builds/updates agentic prompts for daily pulls	Buildable — flagship idea	Briefing Recipe Store + voice-driven LLM recipe refiner with versioned undo + scheduled agentic runner.
Highly robust / self-aware	Buildable as a suite	`self_status`, engine-fallback ledger, nightly self-test email, watchdog sweeper (needs your sign-off vs. the no-alarms decision), canaries, reconnect postmortems, weekly trend report.
Bluetooth remote activation instead of wake word	Yes, with caveats	AVRCP media-button capture + BT car-connect auto-arm (the reliable signal). Media-button routing ambiguity vs. music apps is inherent to Android — managed, not eliminated. HFP-vs-A2DP mic routing is the real quality battle.

Recommended build order

Wave 1 — foundations & quick wins (each S-M): 1. **GitHub client foundation +** `gh_repo_status` — the prerequisite for the entire coding theme. 2. `quota_status` **fuel gauge** — cheapest self-awareness win, all data already exists. 3. `self_status` **tool** — the "how are you doing?" answer. 4. **Bluetooth car-connect auto-arm +**

car-mode defaults (Android) — the reliable activation signal. 5. **Quick Settings tile + shortcuts** — cheapest talk affordance, works outside the car too.

Wave 2 — the coding loop & briefings core (M-L): 6. `code_task_start` / `code_task_status` + **GitHub webhook receiver** + **question relay** — the full remote-coding steering loop. 7. **Briefing Recipe Store + voice recipe tools + LLM refiner** — your prompt-builder. 8. **Briefing runner Lambda + scheduler widening + spoken/email delivery** — daily updates land. 9. **Session foreground service** (Android) — the keystone that keeps conversations alive screen-off; everything car depends on it.

Wave 3 — media & resilience (M-L): 10. `media_control` + `media_play_search` (Android client-local) + audio-focus/ducking contract. 11. **AudioRouteController** — the HFP/A2DP reckoning, verified in your actual car. 12. **Engine health ledger + automatic fallback** + **nightly self-test email**. 13. **Podcasts +** `read_article` **TTS + morning briefing MP3** + TTS cost metering.

Gating realities to respect (verifier-confirmed): the Android app has **never run on a real device** — that verification drive gates every car idea; there is **no FCM and no WebSocket push** (a locked M6 decision — polling + IoT shadow only), so several ideas were re-scoped to polling; **Android settings sync is unshipped** (client-local only); the **watchdog sweeper needs your explicit sign-off** against your no-CloudWatch-alarms decision; and the cheap Gemini engine still awaits your **E1/E2 live-audio verification** before the commute cost-guard idea pays off.

Remote coding sessions from the car (14 suggestions)

Your "ghost-cli" ask, grounded in what this stack can actually do. The pattern that fits best is **kick off, track, steer, land**: a GitHub-integration tool family in the existing server-side tool router. Voice-dictated tasks become GitHub issues with `@claude` mentions (repos with `claude-code-action` installed pick them up and open PRs) or `workflow_dispatch` runs of a headless `claude -p` workflow; a `JOB#` row in DynamoDB tracks each one; a webhook receiver flips the loop from poll to push; and merge-by-voice reuses the exact `send_email` double-gate (server-side allowlist AND spoken confirmation). For repos where Actions is a poor fit, a self-hosted headless Claude Code runner on the existing AWS Batch/Fargate compute environment gives you $0-idle pay-per-task agents.

GitHub client foundation: PAT in SSM + gh_repo_status voice tool

Effort: M · **Impact:** high · **Feasibility:** verified

Add a fine-grained GitHub PAT as SSM SecureString /live-ninja/prod/github/token (set via scripts/set-secret.bat, synced in deploy.yml's secrets step), a small internal/github REST client (go-github or plain net/http), and a first read-only tool `gh_repo_status` returning open PRs, latest workflow run states, and unread review comments for an allowlisted repo set. Tool description instructs the model to answer in <=2 spoken sentences ('live-ninja: main green, 1 open PR awaiting review'). This is the prerequisite plumbing every other GitHub idea reuses.

Prerequisites: Owner creates a fine-grained PAT (repo scope, ideally limited to jeremyproffitt repos) and runs scripts/set-secret.bat; decide which repos are in the allowlist

Risks: PAT is a standing credential in the web fn's reach — scope it fine-grained and read-only for this first tool; token rotation is manual

code_task_start: kick off a Claude Code GitHub Actions run by voice

Effort: M · **Impact:** high · **Feasibility:** verified

SideEffecting tool `code_task_start(repo, prompt, branch?)` that either (a) files a GitHub issue containing the spoken task text plus an @claude mention (repos with anthropics/claude-code-action installed pick it up and open a PR), or (b) fires workflow_dispatch on a `claude.yml` workflow that runs `claude -p` headless with the prompt as input. On success it writes a DynamoDB job row (USER# / JOB##: repo, issue/run id, status=started, ttl 7d) and speaks back a short handle ('Started task 3 on live-ninja'). The car UX is exactly this: dictate a task, get a number, drive on.

Prerequisites: GitHub client foundation; claude-code-action or a claude.yml workflow installed in target repos with ANTHROPIC_API_KEY repo secret (owner sets, agents never see it); repo allowlist

Risks: Voice-dictated prompts are lossy — mitigate by having the tool echo the prompt back for confirmation before creating (two-step confirm like send_email confirmExternal); Actions minutes cost on private repos

code_task_status: polling tool over job rows + GitHub runs API

Effort: S · **Impact:** high · **Feasibility:** verified

Read-only tool `code_task_status(taskId?)` that Queries the user's JOB# partition (no Scan), then hits the GitHub API for live state of each tracked issue/PR/workflow run (queued/in_progress/conclusion, PR opened y/n, checks green y/n), updates the row, and returns a compact structured result the model speaks as one line per task. 'How's my coding task going?' becomes a natural mid-commute check. Also add a conversation.mjs toolTitle/toolFields entry so the web tool card renders task/status/link nicely.

Prerequisites: code_task_start (or at least the JOB# row convention) and the GitHub client

Risks: Low — read-only; stale rows need TTL hygiene (7d ttl covers it)

GitHub webhook receiver: /api/v1/hooks/github with HMAC verification

Effort: M · **Impact:** high · **Feasibility:** verified

New public route (Fiber registrar pattern + authorizer public-allowlist entry) accepting GitHub webhooks (workflow_run, issue_comment, pull_request, check_suite), verified via X-Hub-Signature-256 against a webhook secret in SSM. Handler maps events to the owner's JOB# rows (status transitions, PR URL capture, comment capture) and enqueues an SQS email job for terminal events ('Claude finished live-ninja task 3 — PR #87 opened, checks green'). This flips the model from poll-only to push: the owner hears about completion via email notification on his phone without asking.

Prerequisites: Owner configures the webhook on each repo (or org-level) pointing at live.jeremy.ninja; webhook secret via set-secret

Risks: Public endpoint — must be strict-HMAC-or-404 and never trust payload identity; GitHub delivery retries need idempotent row updates (conditional writes on delivery GUID)

Question relay: Claude asks on the issue, owner answers by voice
Effort: M · **Impact:** high · **Feasibility:** verified

Convention + two small pieces: the Claude Code workflow is prompted to post 'QUESTION: ...' comments on its tracking issue when blocked instead of guessing. The webhook receiver detects those issue_comment events and stores them on the JOB# row as pendingQuestion (and emails a nudge). New SideEffecting tool `code_task_answer(taskId, answer)` posts the spoken reply as an issue comment (with @claude mention to re-trigger the action) and clears the pending flag. code_task_status surfaces pending questions first ('Task 3 is blocked, Claude asks: should timers use UTC?'). This is the core two-way steering loop for the commute.

Prerequisites: Webhook receiver + code_task_start shipped; workflow prompt template amended to enforce the QUESTION comment convention

Risks: Multi-turn confirm in stateless fallback turns is a known open backlog item (plan.md NEXT UP item 6) — in realtime sessions this works today since the model holds context, but composer/fallback answering needs that item; question detection is convention-based, not guaranteed

pr_brief: voice-sized PR summaries (never read the diff)
Effort: M · **Impact:** high · **Feasibility:** verified

Read-only tool `pr_brief(repo, number)` that fetches PR title/body, file-count/±line stats, check-run rollup, review states, and unresolved comment count, then (for the 'what did it actually change' ask) passes the diff summary through a new broker mode `summarize-pr` (clone of extract-topics: gpt-4o-mini, strict JSON, ~3 bullet output) so the spoken answer is '4 files, adds retry backoff to the email dispatcher, tests included, checks green, no review comments' — never raw hunks. Structured card on web shows the real links/stats.

Prerequisites: GitHub client foundation; mirror broker Request/Response structs in webapp api_routes.go (known wire-contract mirror constraint)

Risks: Big diffs must be truncated/stat-only before the LLM call to keep gpt-4o-mini cost trivial; summary quality on huge PRs

pr_merge: merge-by-voice with the send_email double-gate
Effort: M · **Impact:** medium · **Feasibility:** verified

SideEffecting tool `pr_merge(repo, number, confirm)` copying the send_email external-recipient security shape exactly: merge requires BOTH the repo to be on the server-side owner-managed merge-allowlist (fresh DB read, Store.IsAllowed pattern) AND confirm=true which the model may only set after speaking the PR title + check status and hearing explicit assent. Server-side hard gates regardless of confirmation: mergeable_state must be clean and all checks green, squash-merge only, never force. Because push-to-main IS the production deploy for live-ninja, the tool response for that repo appends 'this will deploy to production' to the spoken confirmation.

Prerequisites: GitHub client foundation with a PAT that has contents:write/pull_request:write on allowlisted repos only

Risks: Highest-stakes tool in the catalog — a misheard 'yes' merges to prod; mitigations are the double-gate, green-checks hard requirement, and keeping live-ninja itself OFF the default merge allowlist initially

Delayed check-in: 'check on my coding task in 20 minutes' via EventBridge Scheduler -> Lambda

Effort: M · **Impact:** medium · **Feasibility:** verified with corrections

Extend the one-shot Scheduler pattern beyond email: widen SchedulerRole with lambda:InvokeFunction on a new small `job-check` Lambda (or reuse topics-extract's direct-invoke shape), and add tool `code_task_checkin(taskId, delayMinutes)` that creates a one-shot schedule whose payload names the JOB# row. The Lambda polls GitHub, updates the row, and enqueues a status email — so the owner gets 'Task 3 still running, 12 commits so far' pushed mid-drive without asking. Today set_reminder can already deliver a dumb nudge; this makes the nudge carry live status.

Prerequisites: GitHub client foundation + JOB# rows; template.yaml IAM change (deployed via push-to-main only)

Risks: IAM widening of SchedulerRole must be scoped to the single new function ARN; orphan schedules if a job row is deleted (schedules are self-deleting, low risk)

> **Verifier corrections:** Mechanism is sound but one citation is fabricated: the quoted 'explicitly documented extension point' text ('widening it enables scheduled agent invokes') does NOT exist at template.yaml:1879-1904 or anywhere in the repo — that range is simply the SchedulerRole definition whose only policy is sqs:SendMessage on EmailQueue. Widening it is a real IAM change the reviewer must design, not a documented invitation. Everything else checks out: one-shot at() schedules with ActionAfterCompletion DELETE in internal/tools/scheduler.go:18-23; scheduler:CreateSchedule + PassRole grants template.yaml:293-305; UsageRollupFunction as the new-Lambda clone recipe :604-631; topics-extract as the direct-invoke Lambda shape (cmd/topics-extract, async Event invoke). Scope the new SchedulerRole statement to the single job-check function ARN. Effort M honest.

Morning commute briefing tool: repo_activity digest

Effort: S · **Impact:** medium · **Feasibility:** verified

Read-only tool `repo_activity(sinceHours?)` aggregating across the allowlisted repo set: commits landed, PRs opened/merged, failing workflows, and any pending Claude questions from JOB# rows, returned pre-ranked so the model speaks a 30-second digest ('Overnight: dotfiles PR merged, live-ninja deploy green, one Claude question pending on task 4'). Optionally a second phase adds a weekday-morning scheduled email of the same digest via a rate-schedule Lambda, but the pull version alone fits the 'start of the drive' habit ('Ava, what happened overnight?').

Prerequisites: GitHub client foundation

Risks: Low; N-repo fan-out API calls need a small concurrency cap and 5s-ish budget to stay inside comfortable tool-call latency for voice

workflow_watch: CI/deploy status + rerun-failed by voice

Effort: S · **Impact:** medium · **Feasibility:** verified

Tool pair for the Actions layer specifically: `workflow_status(repo, runId?)` summarizes the latest run per workflow (queued/in-progress/failed step name), and SideEffecting `workflow_rerun(repo, runId, confirm)` re-runs failed jobs after spoken confirmation. Directly useful for live-ninja itself: 'did my deploy finish?' / 'rerun the

failed deploy' from the car, replacing phone-fumbling at red lights. Failed-step names come from the jobs API so the model can say WHICH step died ('deploy failed at Sync secrets to SSM').

Prerequisites: GitHub client foundation with actions:write for the rerun half

Risks: Rerunning a deploy is a production action for this repo — same spoken-confirm discipline as pr_merge

Self-hosted headless Claude Code runner on AWS Batch (Actions-free alternative)
Effort: L · **Impact:** medium · **Feasibility:** verified

For repos/tasks where GitHub Actions is a poor fit (long jobs, no action installed, private experimentation): a containers/code-runner/ image (git + node + claude CLI) run as a second Batch job definition on the existing Fargate compute environment. `code_task_start` gains a runner=batch option: web fn batch:SubmitJob with containerOverrides env carrying the prompt + repo + JOB# id; the container clones via the PAT, runs `claude -p --output-format json`, streams progress heartbeats to the JOB# row, pushes a branch, and drops logs/outputs into the deliverables bucket (so `file_read`/Download Center can inspect them later). ANTHROPIC_API_KEY lives in SSM, injected as a Batch job env from the job role. $0 idle, pay-per-task, 20-min-style timeout knob.

Prerequisites: ANTHROPIC_API_KEY + PAT as SSM params granted to the Batch job role; container build; JOB# row convention from code_task_start

Risks: An agent with a write-capable PAT running unattended in your VPC — constrain to push-branch-only (never main), egress is open (no NAT but public IPs) so the PAT scope is the real boundary; Fargate per-minute cost is real but bounded by timeout; biggest effort item on this list

Live session push: coding events spoken mid-conversation
Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

When a webhook event lands while a voice session is active (ACTIVEUSER marker is already written on transcript posts), surface it in-session instead of only by email: web/Android clients get the event over the existing settings-fanout channels (web WebSocket /v1/ws, Android FCM) as a new additive message type, and the client injects a text turn ('[system] Task 3 finished: PR #87 opened') via the existing sendUserText path so the assistant announces it naturally. Falls back to email when no session is live. This is the 'Claude finished while you were mid-conversation about dinner plans' magic moment.

Prerequisites: Webhook receiver shipped; additive WS/FCM message type (contracts additive-only rule — amend contracts first per contracts/README.md)

Risks: Client work on two surfaces; interrupting the model mid-response needs care (queue events to turn boundaries); Android FCM leg untested until the app finally runs on a real device (known open item)

> **Verifier corrections:** The cited fan-out channels DO NOT EXIST in code — this is the one idea whose foundation is spec-fiction. internal/sync/sync.go's header records the locked M6 decisions: 'No FCM' (would require a Firebase account, declined) and 'No WebSocket API: cost/complexity not justified' — web reconciles settings by POLLING GET /api/v1/settings?since= every 30s; conversation.mjs:1421 states outright 'the web WebSocket/settings.updated frame does not exist — only the device shadow path has push'. /v1/ws lives only in contracts/api.md as unbuilt spec. Additionally, ACTIVEUSER (api_routes.go:754-757, store/usage.go:360) is a DAY-granular CONFIG-partition marker with 48h TTL written on transcript posts — it says 'active today', not 'in a session right now', so it cannot route a webhook

> to a live session. What IS real: sendUserText (realtime.mjs:1974, used by the composer at conversation.mjs:1208) — injecting a system text turn into a live session works today. Feasible re-scope: during an active session the client polls a small pending-events endpoint (piggyback the existing 30s settings-poll cadence or the transcript-post loop) fed by the webhook receiver's JOB# rows, and injects via sendUserText at turn boundaries; email remains the no-session fallback. As-written (WS+FCM) the effort is not L — it's building two push subsystems the project already explicitly declined; the polling re-scope keeps it ~M for web-only.

gh_issue_capture: drive-by idea/bug filing into any repo
Effort: S · **Impact:** medium · **Feasibility:** verified

Small SideEffecting tool `gh_issue_create(repo, title, body, labels?)` so commute thoughts become tracked issues ('file an issue on live-ninja: settings drawer loses scroll position'). Pairs naturally with code_task_start (an issue filed now can get an @claude mention tonight from the desk, or immediately via a followup 'and have Claude take a crack at it'). Cheap, low-risk, and exercises the whole GitHub foundation; also the natural home for the existing remember_note overflow — notes that are really work items.

Prerequisites: GitHub client foundation with issues:write

Risks: Minimal; dictation quality of issue bodies (model should compose the body from context, not transcribe verbatim)

Persona: 'Devin' the engineering copilot persona tuned for car coding
Effort: S · **Impact:** medium · **Feasibility:** verified

A built-in persona whose style block is tuned for this workflow: terse status-first replies, always leads with counts and states not narration, never reads code/diffs/URLs aloud, proactively offers next actions ('want me to merge it or wait?'), and knows the task vocabulary (task numbers, checks, PRs). Costs one builtinDef entry plus a Gemini voice mapping, and makes every tool above measurably better over Bluetooth. Ship it alongside the first two tools so the car UX is coherent from day one.

Prerequisites: None — independent, but pointless before at least gh_repo_status exists

Risks: None meaningful

Media playback (13 suggestions)

"Play media / play videos" in a car means audio-first, and the honest architecture is: **let native apps do the playing, let Live Ninja do the controlling.** On Android, `media_control` tools execute client-locally as media-key events, and `media_play_search` fires the same intent Google Assistant uses to make YouTube Music/Spotify start playback with zero API keys — audio routes to the car natively. Server-side, a keyless podcast tool family (iTunes Search + RSS), long-form article TTS through the cheap `fallback-tts` leg (roughly 10x cheaper than having the realtime model read aloud), and a generated morning-briefing MP3 in your Download Center cover owned content. An audio-focus/ducking contract keeps assistant and media from fighting.

media_control tool family with Android-local execution (play/pause/skip/volume on the phone's active media app)

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

Add media_play, media_pause, media_next, media_previous, media_volume as tool Definitions in internal/tools/ + the toolManifest mirror in internal/realtime/mint.go, but execute them CLIENT-LOCALLY on Android: intercept these tool names in ToolCallRouter before the generic POST /api/v1/tools/invoke forward, and dispatch KeyEvent media-button presses via AudioManager.dispatchMediaKeyEvent (no special permission) or MediaSessionManager transport controls (needs NotificationListener grant, richer state). Server-side handlers return a structured 'executes on device' result for non-Android surfaces (mirroring the Tab5 'not available on this device' precedent in gemini-plan.md §10). This is the single highest-value car feature: 'pause the music', 'skip this song' while driving over Bluetooth, controlling whatever app (YouTube Music, Spotify, podcast app) is already playing.

Prerequisites: Amend contracts/api.md tool catalog first (additive-only). Decide client-local vs server-roundtrip execution semantics: cleanest is Android answering the functionCall directly and still POSTing a tools/invoke audit record, or a new 'deviceLocal:true' flag in the manifest so all engines' declarations stay generated from one place (gemini_mint.go geminiToolDeclarations and fallback_tools.go derive automatically).

Risks: MediaSessionManager needs BIND_NOTIFICATION_LISTENER_SERVICE user grant (must no-op gracefully per the optional-grant convention); media KeyEvents are fire-and-forget with no confirmation of which app handled them; Android app has never been run on a real device yet (open backlog item) so this stacks on unverified ground.

> **Verifier corrections:** Anchors all verified: android/app/src/main/java/ninja/jeremy/liveninja/realtime/ToolCallRouter.kt is exactly the claimed never-throws dispatcher with callId-as-idempotency-key; RealtimeSessionCoordinator.kt:180/201 routes RealtimeEvent.FunctionCall through it for all engines, so one interception point covers OpenAI/Gemini/Nova. internal/tools/registry.go NewRegistry (:282) slice and internal/realtime/mint.go toolManifest (:117, 20 tools) exist, and the 'declarations generated from one place' claim is true — gemini_mint.go:121 and fallback_tools.go:64 both derive from toolManifest (with sync tests). Tab5 precedent confirmed: gemini-plan.md:548 and firmware/components/ln_realtime/ln_realtime.c:603 emit 'tool execution is not available on this device'. REQUIRED CHANGES: (1) client-local execution contradicts a documented invariant in TWO places — mint.go:110-116 ('Execution never happens client-side...') and contracts/api.md:61 ('/v1/tools/invoke — Server-side tool execution') — both must be amended, not just api.md's catalog; (2) because fallback-turn (server-side chat-completions) shares the manifest, the server handler returning a structured 'executes on device' result is mandatory, not optional, or fallback text turns wedge. Effort M is honest. Risk note is accurate: plan.md:616/:631 confirm Android has NEVER run on a device/emulator.

media_play_search: 'play X' launches YouTube Music/Spotify via INTENT_ACTION_MEDIA_PLAY_FROM_SEARCH

Effort: S · **Impact:** high · **Feasibility:** verified

A media_play_search(query, app?) tool that, on Android, fires MediaStore.INTENT_ACTION_MEDIA_PLAY_FROM_SEARCH (the exact intent Google Assistant uses) with EXTRA_MEDIA_FOCUS hints, optionally pinned to a package (com.google.android.apps.youtube.music, com.spotify.music). The target app resolves the query and starts playback with zero API keys, zero OAuth, zero server cost — playback stays entirely in the native app over Bluetooth. Combined with the media_control family

above, this covers 'play some jazz', 'play the latest Lex Fridman episode on YouTube Music', 'skip'. Answers the owner's 'playing videos' ask in the only car-safe way: YouTube/YT Music app plays it, audio routes to the car, and control comes back through media buttons.

Prerequisites: media_control idea's client-local execution seam (or ship together as one wave). App picker default should live in settings (contracts/settings.schema.json additive field, e.g. mediaApp enum) so it follows the populated-control UX rule.

Risks: Background YouTube video audio requires YouTube Premium (YT Music does not); intent behavior varies by app version — needs real-device verification, which is already a gating backlog item for Android overall.

Audio-focus + ducking policy: assistant and media coexist in the car
Effort: M · **Impact:** high · **Feasibility:** verified

Define and implement the coexistence contract on Android: (1) WakeWordService keeps detecting during media playback (it already runs an independent mic FGS with EnergyVad); (2) on wake or PTT, request AudioManager focus AUDIOFOCUS_GAIN_TRANSIENT_MAY_DUCK for assistant speech so music ducks, escalating to a media_pause KeyEvent when a realtime session actually opens (mic capture + music = garbage ASR in a car cabin); (3) on session end/keep-warm expiry, abandon focus and optionally auto-resume media. Mirror the pattern already proven on web: realtime.mjs patient-mode ducks remote audio to 0.15 gain for barge-in — this is the same idea pointed at external media. Emit RealtimeEvent lifecycle transitions from RealtimeSessionCoordinator as the trigger points so transports need no changes.

Prerequisites: media_control tool family (for pause/resume verbs); a settings knob (duck vs pause during assistant turns) as an additive settings.schema.json field

Risks: AEC on VOICE_COMMUNICATION source may not fully cancel car-speaker music, causing echo-triggered self-barge-in (a known rough edge on Tab5, plan.md §8 item 10) — pause-during-listening is the safe default; focus behavior differs across OEMs/Bluetooth stacks.

Podcast tool family: podcast_search / podcast_latest via iTunes Search + RSS (keyless, serverless)
Effort: M · **Impact:** high · **Feasibility:** verified

Server-side tools podcast_search(query) and podcast_episodes(feedUrl|id) that hit the keyless iTunes Search API (podcast entity) and parse the RSS feed with encoding/xml to return episode titles, dates, durations, and enclosure MP3 URLs. Follows exactly the get_weather (Open-Meteo, keyless) and web_lookup (Wikipedia) pattern: narrow Deps interface, structured parser, no API key, no standing infra, media bytes never touch AWS (the client fetches the public enclosure URL directly — preserves the media-path invariant's spirit). The model can then answer 'what's the latest episode of Oxide and Friends' and hand the URL to a playback tool (companion player idea below, or media_play_search on Android).

Prerequisites: Manifest entries in mint.go toolManifest; a web tool-card entry in conversation.mjs toolTitle/toolFields for pretty episode cards

Risks: RSS is messy in the wild (encodings, malformed enclosures) — bound fetch size (~1MB like research.go) and fail per-feed; some enclosure URLs are tracker-redirect chains needing the redirect-hop re-validation treatment.

Companion audio player in the web client with assistant-aware ducking (media_queue/media_playback tools)

Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

A mediaplayer.mjs module on /conversation: an HTMLAudioElement (or fetch+MediaSource) routed through a WebAudio GainNode — the exact duckable-playback architecture realtime.mjs already uses for remote assistant audio — playing podcast enclosures, deliverable MP3s (briefings/TTS articles below), and any direct URL. It subscribes to the existing RealtimeSession public event surface (speechstarted/speaking/bargein/sessionready) to duck to 0.15 on assistant speech and pause on user speech, with MediaSession API metadata so browser/OS media keys work. Server-side media_queue_add/media_queue_status tools persist queue+position as a MEDIA# item under USER# (single-partition Query, additive SK prefix per PRD §8.1) so 'resume my podcast' survives page reloads and, later, surfaces cross-device.

Prerequisites: Podcast tool family (content source). CSP note: media-src for cross-origin enclosure URLs must be added in internal/webapp/pages_routes.go (connect-src is currently 'self'+OpenAI+Gemini;

Risks: The car surface is Android, not web — this is desk-first; keep it thin. Autoplay policy: playback must start inside a user/tool gesture chain (same constraint realtime.mjs documents).

> **Verifier corrections:** #novaEnqueueAudio GainNode path confirmed (realtime.mjs:998) and the one-EventTarget event surface exists. CSP claim verified with a correction: internal/webapp/pages_routes.go:51-52 — media-src is currently 'self' blob: so it MUST be widened for cross-origin enclosures (arbitrary podcast hosts means media-src https:, a real policy loosening to own); connect-src is actually 'self' + api.openai.com + wss generativelanguage + two wakeword S3 hosts (not just 'self'+OpenAI+Gemini). The idea's 'verify' resolves as: plain HTMLAudioElement needs only media-src, but the mentioned fetch+MediaSource variant WOULD require connect-src widening — pick HTMLAudioElement. MEDIA# SK addition matches the NOTE#/GUIDE# single-partition convention (no Scan). Effort L honest; desk-first caveat is correct.

read_article: long-form TTS reading through the cheap fallback-tts leg instead of the realtime engine

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

A read_article(url) tool: server fetches the page (extend the research.go fetch leg with a readability-style extraction and a broader-but-guarded domain policy), chunks the text, and synthesizes speech via the existing broker fallback-tts mode (gpt-4o-mini-tts) — roughly 10x cheaper per audio-minute than having the realtime model read text aloud, which matters directly against the ~$15/mo quota ceiling. Output: either (a) chunked audio streamed to the companion player, or (b) simplest serverless shape — write an MP3 per article into the deliverables bucket via internal/deliv and return a presigned URL for the client player. 'Read me that Hacker News article' on the commute becomes a one-tool flow, and the realtime session stays open for barge-in questions while the article plays through the ducking path.

Prerequisites: Decide the SSRF posture for arbitrary-article fetch (current direct-fetch allowlist is only anthropic.com/openai.com — likely a per-request owner-visible domain policy or a broader readable-content allowlist); TTS chunk length limits per OpenAI TTS API; Lambda 15-min cap bounds article length (fine for articles, not books).

Risks: Long articles = many TTS calls = cost that must be metered (route through CheckFallback and record into USAGE# like fallback turns); extraction quality varies — fail loudly with a structured ToolError rather than reading

nav-bar soup.

> **Verifier corrections:** Anchors verified: broker fallback-tts mode (cmd/realtime-broker/main.go:276, handleFallbackTTS:586, CheckFallback gate:636; web route api_routes.go:78); internal/deliv exists; 180-day deliverables lifecycle (template.yaml:1315-1319) and 15-min presigned GET (deliverables_routes.go:8) confirmed; current direct-fetch allowlist is indeed only anthropic.com/openai.com (research.go:46) so the SSRF-posture decision is genuinely open and load-bearing. CHANGES: fallback-tts today is a single request/response returning MP3 to the caller — a chunk-loop composing into a deliverable is new server plumbing; the worker→broker direct-invoke precedent it needs exists (topics-extract → broker extract-topics, template.yaml:731-736). Metering: day-level USAGE# items are written in real time and cmd/usage-rollup derives monthTokens, so TTS spend must write day-token records itself (the idea's risk section already says this). Effort L honest.

Morning commute briefing: scheduled generated audio (NotebookLM-lite) dropped into the Download Center

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

A rate-based scheduled Lambda (clone the UsageRollupFunction Events.Schedule shape, cron ~7am ET) that composes a personal briefing — weather via the weather tool path, calendar-free for now, open plan/task items from the memory layer (PLAN#/task entities), yesterday's conversation topics (CONV#/TREF#), top HN items via the research.go Algolia leg — scripts it through the broker fallback-turn (gpt-4o-mini), synthesizes via fallback-tts, and writes briefing-YYYY-MM-DD.mp3 into the deliverables bucket. A play_briefing tool (or the companion player auto-surfacing today's file) plays it in the car; optionally an SES email with the presigned link via the existing EmailQueue. Zero standing infra: one cron Lambda, S3, existing broker modes.

Prerequisites: read_article's TTS-to-deliverable plumbing is 80% shared — build that first and this becomes the scheduled composition on top. Broker needs a server-invoke path for TTS from a worker (topics-extract -> broker extract-topics is the exact precedent for worker->broker invocation).

Risks: gpt-4o-mini-tts single-voice monologue, not NotebookLM two-host dialogue — set expectations (a two-voice script alternating TTS voices is a cheap upgrade); briefing generation cost is small but should still write USAGE# tokens.

> **Verifier corrections:** UsageRollupFunction Events.Schedule confirmed at template.yaml:604-617 (Schedule: rate(1 hour); a 7am ET run needs a cron() expression — trivially supported). CORRECTIONS to buildsOn: (1) topics have NO GSI3 — they are CONV#/TOPICREF#/TOPIC# items under the user partition with a deliberate no-new-GSIs decision (internal/store/topics.go:5, template.yaml:735), and the prefix is TOPICREF# not TREF#; (2) plans/tasks are NOT PLAN# items — they are ENT#plan#/ENT#task# entities (internal/store/entities.go entSK); reads still fit single-partition Query, so the no-Scan rule holds either way. EmailQueue + email-dispatch confirmed (template.yaml:636-651). Worker→broker invoke precedent confirmed (topics-extract). Correctly sequenced behind read_article's TTS-to-deliverable plumbing. Effort L honest; single cron Lambda + S3 fits no-standing-infra.

'Read my stuff' tools: read_note / read_deliverable / read_plan routed through TTS

Effort: M · **Impact:** medium · **Feasibility:** verified

Thin verbs over content the platform already stores: 'read me my notes from yesterday', 'read the deliverable you made this morning', 'what's on my plan — read it out'. Implementation is a read_aloud(source, id) tool that resolves via existing recall_note / file_read / plan lookups, then reuses the read_article TTS pipeline (chunk ->

fallback-tts -> deliverable MP3 or streamed chunks). Short content (<~1 min) can skip TTS entirely and just return text for the realtime model to speak — the tool result can carry a speakDirectly hint and the estimated audio cost so the model (or a length threshold server-side) picks the cheap path automatically. This turns the existing notes/deliverables/plans corpus into commute-consumable audio with near-zero new surface area.

Prerequisites: read_article TTS pipeline. Manifest + gemini/chat-completions derivations come free once registered in both registry.go and mint.go toolManifest.

Risks: Low; main risk is over-tooling — could alternatively be a single read_aloud tool with a source enum to keep the 20-tool catalog from sprawling (enums grow, never shrink).

Playback state in the settings/shadow sync fabric: cross-surface 'resume where I left off'

Effort: M · **Impact:** medium · **Feasibility:** verified with corrections

Persist now-playing state (source, episode/URL, position, queue) as a versioned MEDIA#state doc updated on pause/duck/interval, and fan out via the existing cross-surface sync: web WebSocket /v1/ws, Android FCM, and (later) the Tab5 config shadow. 'Resume my podcast' in the car picks up where the desk web player stopped. Reads/writes follow the settings optimistic-concurrency + unknown-field round-trip rules; a media_queue_status tool exposes it to the model so 'what am I listening to?' and 'play the next one' work from any surface.

Prerequisites: Companion player and/or Android playback integration writing position updates; decide write cadence (on state change + 30s interval max) to respect on-demand DynamoDB cost discipline

Risks: Position tracking is only possible for playback Live Ninja itself renders (companion player); native-app playback via intents is opaque — state doc must honestly model 'delegated to ' with no position.

> **Verifier corrections:** MAJOR CORRECTION: the claimed fan-out fabric does not exist. There is NO web WebSocket /v1/ws and NO Android FCM — internal/webapp/settings_routes.go:88-95 documents the locked M6 decision that REPLACED the 'WebSocket/FCM push sketch (no Firebase account; no WebSocket API)' with polling: web polls ?since= every 30s + visibilitychange/focus, Android polls on foreground + a 15-min wake-service tick; the only real push surface is the IoT device shadow (settings_routes.go:158-180, contracts/shadow.md). So the design must be: MEDIA#state versioned doc + optimistic-concurrency 409 (that pattern genuinely exists in settings) with settings-style ?since polling — which is actually fine for 'resume where I left off' (resume is a session-start read, not a live push need). With that reframing the idea works and effort M stands. The 'delegated to , no position' honesty note is correct.

Barge-in contract for media: wake word stays hot during playback, one policy across engines

Effort: M · **Impact:** high · **Feasibility:** verified

Codify (in docs/voice-engines.md + implement) the interaction matrix between media playback and the three voice engines, because their cancel primitives differ: OpenAI supports server interrupt (<=150ms mirror), Gemini has NO server cancel (local flush only), Nova is bridge-mediated. Policy: media playing + wake -> pause media locally BEFORE opening the session (avoids the echo-triggered self-barge-in class of bugs); assistant speaking + media request -> assistant finishes or is barged, then playback starts; assistant speaking over active media -> duck via the GainNode/AudioFocus paths. Implement as a small MediaCoordinator on each client that owns the ordering, driven by the existing normalized event surfaces (web RealtimeSession events, Android RealtimeEvent stream) so no transport changes are needed.

Prerequisites: At least one playback path shipped (companion player or Android media_control); this is the glue that keeps them from fighting

Risks: Mostly design discipline; the failure mode it prevents (music leaking into ASR, self-barge-in) is already documented on Tab5, so skipping this idea taxes every other media idea.

Podcast feed watcher: subscriptions + new-episode prefetch for flaky-connectivity commutes

Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

podcast_subscribe/unsubscribe tools writing FEED# items, plus an hourly scheduled Lambda (second Events.Schedule cron, or piggyback a new schedule on a dedicated fn) that re-fetches subscribed RSS feeds, diffs episode GUIDs, writes EPISODE# metadata rows (TTL 30d), and optionally pre-copies the newest enclosure MP3 into the deliverables bucket so the car client streams from CloudFront/S3-presigned instead of a slow origin over cell. 'Any new episodes?' becomes a single-partition Query; new-episode notification can ride the existing EmailQueue or FCM fan-out. Storage cost is bounded by the existing 180-day deliverables lifecycle; prefetch is opt-in per feed.

Prerequisites: Podcast tool family shipped; a per-feed prefetch flag in the FEED# doc; Lambda egress cost check for large MP3 copies (50-100MB episodes through Lambda memory — stream with io.Copy, the deliverables-zipper already streams S3 this way)

Risks: Copying third-party audio into your S3 is fine for personal single-owner use but is the one idea that puts media bytes on AWS — keep it opt-in; hourly polling of N feeds is pennies but should carry the Project cost tags story.

> **Verifier corrections:** Events.Schedule cron pattern, DynamoDB TTL usage, deliverables 180-day lifecycle, presigned download route (deliverables_routes.go), and EmailQueue all verified. Streaming-copy precedent confirmed: internal/deliv/zipjob.go streams S3 through io.Copy (:120) without buffering whole objects. CORRECTION: the FCM notification leg does not exist (no Firebase account — settings_routes.go:93); notification options are EmailQueue or nothing/poll. FEED#/EPISODE# single-partition items fit the no-Scan rule, but note the hourly Lambda must enumerate subscribed feeds via a Query under USER# or a CONFIG-partition marker (the ACTIVEUSER# pattern in cmd/usage-rollup is the precedent) — never a table Scan; single-owner reality makes this trivial. The media-bytes-on-AWS caveat is honestly flagged and opt-in. Effort L honest.

Tab5 ambient audio playback (podcast/briefing on the desk terminal)

Effort: XL · **Impact:** low · **Feasibility:** verified with corrections

Extend ln_realtime/ln_audio so the Tab5 can play non-conversation audio: the 24kHz downlink jitter-ring path in ln_audio is format-identical to what a briefing/TTS MP3 decoded to PCM would need, so the plausible shape is a new ln_media task that HTTP-streams a deliverable (WAV/PCM served in a device-friendly encoding from a new ?format=pcm24k transcode option on the deliverable download route, avoiding an on-device MP3 decoder) into the existing jitter ring, with LN_UI playing/paused screens and wake-word barge-in flushing via the existing play_stop path. Honest assessment: this is the most expensive surface for the least car value — include it only because the briefing/deliverable audio corpus will exist anyway and the audio plumbing genuinely lines up.

Prerequisites: Briefing/TTS deliverables shipped; server-side PCM transcode option (or ship MP3s as WAV alternates at generation time — cheaper than firmware codec work); Tab5 pairing e2e (ProvisionIoT stub) still

open, which gates all device work

Risks: PSRAM/task budget on ESP32-P4 during long playback; SDIO internal-DMA-heap issue is an open rough edge; strongly suggest deferring until car-side ideas ship.

> **Verifier corrections:** Firmware anchors verified: ln_audio.h documents exactly the claimed path — 24 kHz mono pcm16 downlink → halfband upsample → jitter/ring buffer with prebuffer config (:12-14), and ln_audio_play_stop() exists (:61); ln_net/src/ln_backend.c and ln_auth_get_jwt (ln_auth.c) exist for authenticated fetch. CORRECTION (stale gating claim): the Tab5 e2e voice loop was HIL-VERIFIED on hardware 2026-07-19 (plan.md:563 M14 item 8: wake → mint → WSS → audible answer), so device work is NOT wholly gated; what remains stubbed is only the ProvisionIoT IoT Thing/cert leg (plan.md:631) — that gates shadow/MQTT features, not JWT-authenticated HTTPS deliverable fetch, which is the path this idea needs. Known rough edges (SDIO internal-DMA heap, uplink shedding during downlink — plan.md:566) are real and make long playback risky. Effort XL / impact low is an honest self-rating; the defer-until-car-ideas-ship recommendation is correct.

Cost surfacing for media/TTS: rates.go entries + badges so audio generation is metered like everything else

Effort: S · **Impact:** medium · **Feasibility:** verified

Small but load-bearing given the owner's cost posture: add gpt-4o-mini-tts per-character/per-minute rates to internal/realtime/rates.go (keyed by model id per the existing rule), meter every read_article/briefing synthesis into the USAGE# month bucket via the same bookkeeping as fallback turns, and surface 'audio generated: $0.04' on the tool result cards and the monthly cost drawer line. Prevents the TTS features above from becoming an unmetered side-channel around the ~$15/mo quota ceiling, and keeps the cost badge honest.

Prerequisites: Ships alongside the first TTS-generating feature (read_article or briefing)

Risks: None significant; omitting it is the risk.

Daily briefings & the prompt-recipe builder (15 suggestions)

Your "tool to help build and update agentic prompts to pull data like a daily update" maps to a **Briefing Recipe Store**: server-side recipe documents (prompt text + typed sources + schedule + delivery flags) that you create and refine *by voice* ("add crypto prices to my morning brief") through an LLM refiner mode with versioned history so "undo that change" works. A runner Lambda executes recipes through the existing agentic tool loop, schedules ride EventBridge Scheduler (pay-per-invocation, zero standing cost), and delivery fans out to spoken-at-session-start, email, and deliverable files. Note: your message truncated at "grab news and updates from d." — the generic `fetch_feed` source plugin plus the GitHub digest covers news feeds, dev updates, and arbitrary sources; tell me what "d." was and it slots in as one more source type.

Briefing Recipe Store: RECIPE# entity + CRUD API

Effort: M · **Impact:** high · **Feasibility:** verified

The foundation: a server-side 'briefing recipe' document — {id, name, promptText (the agentic instruction), sources[], schedule (cron/at expression or null), delivery: {spoken, email, deliverable}, enabled, version, lastRunAt, history[] of prior promptText revisions}. Stored at USER#/RECIPE# in the single table, additionalProperties:true

with unknown-field round-trip per contracts rules. REST at /api/v1/briefings/recipes (GET/PUT/DELETE) following the RegisterXxxRoutes registrar pattern, plus a contracts/briefing.schema.json amended FIRST per contracts/README.md. Cap ~20 recipes, promptText capped ~4000 chars (mirrors the guides 20/6000 discipline).

Prerequisites: None — pure additive entity + routes

Risks: Schema churn later if delivery/source shapes aren't designed additively up front; keep sources[] as typed-but-open objects {type, params{}}

Voice-driven recipe editing: briefing_upsert / briefing_list / briefing_run tools

Effort: M · **Impact:** high · **Feasibility:** verified

Expose recipes to the voice model so the owner can say 'add crypto prices to my morning brief' mid-drive. Three tools in internal/tools/: briefing_list (names + one-line summaries), briefing_upsert (SideEffecting, idempotency-guarded; accepts either full recipe fields or a natural-language 'change' instruction), briefing_run (trigger now, async). Register in NewRegistry (registry.go:303-324) AND mirror in the realtime toolManifest (mint.go:117) — the map explicitly warns tools missing from the manifest are invisible to the voice model. Gemini/fallback views derive automatically.

Prerequisites: Recipe store (idea 1)

Risks: toolManifest/registry drift is a known repo gotcha — add both in one commit; keep param schemas enumerated so validation stays strict

LLM recipe refiner: 'update my brief' rewrites the prompt server-side with versioned diffs

Effort: M · **Impact:** high · **Feasibility:** verified

The prompt-builder itself: a new realtime-broker mode refine-recipe (clone the extract-topics mode shape) that takes {currentPromptText, sources, userInstruction} and returns a rewritten recipe via gpt-4o-mini with strict-JSON output + server-side sanitization, exactly like realtime/extract.go does for topics. briefing_upsert calls it when given a natural-language change; the old promptText is pushed onto history[] (last ~10 revisions) so 'undo that change to my brief' works by voice. The spoken confirmation reads back a one-line diff summary ('Added a crypto prices section after weather').

Prerequisites: Ideas 1-2

Risks: Prompt-injection via userInstruction into the standing recipe prompt — frame recipe prompts as data under an immutable runner core (same ComposeCustomInstructions tone-only framing personas use); extract-topics is ungated but this should pass CheckFallback quota

Briefing runner: agentic executor Lambda reusing the fallback tool loop

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

A cmd/briefing-runner direct-invoke Lambda (topics-extract shape) that loads a recipe, then runs the EXISTING agentic loop: realtime/fallback_tools.go TurnWithTools already does gpt-4o-mini + the full 20-tool catalog with a 5-iteration tool loop — the runner is that loop pointed at recipe.promptText with web_research/get_weather/fetch_feed etc. available, iteration cap raised to ~10 for multi-source pulls. Output: a

structured brief {sections[], spokenSummary (~90s of speech), fullMarkdown, costEstimate} written to USER#/BRIEF# with 7d TTL. This is the single biggest new component and everything else (schedule, delivery, commute) hangs off it.

Prerequisites: Ideas 1; runner needs the web fn's tool IAM (or invoke tools via web fn) — cleanest is running the loop INSIDE the web function triggered async, keeping broker key-isolation intact (web asks broker for the LLM turn, executes tools locally, exactly like fallback/turn does today)

Risks: Lambda 15-min cap fine, but per-run token cost needs a gate (idea 12); tool loop against live web sources can be slow — bound per-source timeouts

> **Verifier corrections:** Two corrections. (1) TurnWithTools (fallback_tools.go:122) is a SINGLE model turn that 'never executes anything itself' — the 5-iteration execute loop lives in the web function at api_routes.go:912 (maxFallbackToolIterations=5 at :856), executing via the registry. The reusable asset is that loop's shape, not TurnWithTools alone. (2) 'Run the loop INSIDE the web function triggered async' fights the WebFunction's 30s global timeout (Globals Timeout: 30, template.yaml:100) and the Lambda-Web-Adapter HTTP event model — multi-source agentic runs won't fit. Right shape is a dedicated direct-invoke Lambda (cmd/topics-extract clone, Timeout 300 like deliverables-zipper/account-purge) that calls the broker for LLM turns (key isolation preserved) and executes tools locally with its own tools.Deps wiring + IAM (SQS email, scheduler, S3 deliverables, DDB, iot). That IAM/deps duplication confirms effort L is honest, maybe L+.

Scheduled execution: widen SchedulerRole to invoke the briefing runner

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

Recipes with a schedule get a real EventBridge Scheduler schedule in the existing live-ninja group. Two template.yaml changes: (a) SchedulerRole (template.yaml:1879-1904) gains lambda:InvokeFunction on the runner (today its ONLY permission is sqs:SendMessage to the email queue — the capability map flags this exact widening as the extension point); (b) web fn's existing scheduler:CreateSchedule + PassRole (template.yaml:291-305) is reused as-is. briefing_upsert creates/updates/deletes the schedule alongside the recipe (cron for daily, weekdays-only supported: 'cron(30 6 ? * MON-FRI *)' for the commute). Zero standing cost — Scheduler is pay-per-invocation, matching the cost posture.

Prerequisites: Idea 4 (runner)

Risks: IAM change = template.yaml change = production deploy on push; schedule lifecycle must be cleaned up on recipe delete and account purge (add a step to cmd/account-purge)

> **Verifier corrections:** SchedulerRole confirmed at template.yaml:1879-1904 with exactly one permission (sqs:SendMessage to EmailQueue) as claimed; web fn CreateSchedule+PassRole confirmed at :293-305; scheduler.go builds one-shot at() expressions (:115-120) so a cron+Lambda-target variant is a small delta. Missing from the idea: the web fn's Sid TimerSchedules grants ONLY scheduler:CreateSchedule — recipe update/delete lifecycle also needs scheduler:UpdateSchedule/DeleteSchedule/GetSchedule added, plus the account-purge step it does mention (cmd/account-purge exists). Zero standing cost claim is correct. Effort M honest including the IAM edits; correctly flags that template.yaml change = prod deploy on push.

Generic source plugin: fetch_feed tool (RSS/Atom/JSON Feed) with per-recipe allowlist

Effort: M · **Impact:** high · **Feasibility:** verified

Covers the truncated 'grab news and updates from d...' generically: a fetch_feed tool that pulls and parses RSS/Atom/JSON-feed URLs, returning {title, link, published, summary} items (top N, since-timestamp filter). SSRF discipline copied from research.go: https-only, redirect-hop re-validation, 1MB/10s caps — but instead of a hardcoded domain list, the allowlist is the set of source URLs saved in the owner's recipes (server-side read, so the model can only fetch feeds the owner explicitly added by voice/UI: 'add Hacker News RSS to my brief' -> briefing_upsert stores the URL -> fetch_feed may fetch it). Use Go's encoding/xml with a small feed struct — structured parser per house rules, no scraping.

Prerequisites: Idea 1 (allowlist source of truth)

Risks: Feed HTML-in-summary needs sanitizing before it reaches the LLM/spoken path; malformed feeds — fall back to item-title-only

GitHub source: github_digest tool (notifications, PR/CI status) via PAT in SSM
Effort: M · **Impact:** high · **Feasibility:** verified with corrections

Dev updates leg: a github_digest tool hitting api.github.com REST (notifications, involved-me issues/PRs, latest workflow runs for named repos incl. live-ninja's own deploy.yml status) with a fine-grained PAT. Secret flows the sanctioned way: GitHub secret set via scripts/set-secret.bat -> deploy.yml 'Sync secrets to SSM' step -> SSM SecureString /live-ninja/prod/github/token -> ssm:GetParameter + kms:Decrypt ViaService=ssm grant on the web fn (exact pattern at template.yaml:244-258). Returns compact structured items so the brief can say 'CI green on live-ninja, 2 PRs awaiting your review'.

Prerequisites: Owner runs set-secret for the PAT (agents never see the value)

Risks: PAT scope creep — restrict to notifications+actions:read; rate limits trivial at this volume

> **Verifier corrections:** Secret flow verified end-to-end: deploy.yml 'Sync secrets to SSM' step at :283-327 with SecureString put-parameter calls; scripts/set-secret.bat exists; config.Loader 5-min in-memory SSM cache confirmed (internal/config/config.go:49 cacheTTL). One correction: the SSM grant at template.yaml:244-251 (Sid LwaParams) is scoped to parameter/live-ninja/prod/lwa/ *only — a /live-ninja/prod/github/token param needs its own resource line added (the KMS ViaService=ssm decrypt at :251-258 is already resource:* so it covers it), plus a new entry in the deploy.yml sync step and a new GitHub secret. All routine edits; pattern claim otherwise exact. Effort M honest.

Spoken delivery: brief-aware session start + read_briefing tool
Effort: S · **Impact:** high · **Feasibility:** verified

When a fresh unheard BRIEF# exists, the mint pipeline appends a short server-derived directive to instructions — exactly how guides.go injects standing directives today: 'A morning briefing from 6:35am is ready; offer to read it.' Plus a read_briefing tool returning spokenSummary so any engine (OpenAI/Gemini) can deliver it, and heardAt is stamped so it's offered once. Zero client changes on any surface — this rides the existing instruction-composition and tool-invoke paths, so it works identically on web, Android, and Tab5.

Prerequisites: Idea 4

Risks: Instruction budget — keep the injected cue to one line, never the whole brief (the tool fetches the body); broker mirrors another SK constant (known accepted pattern, personas_store.go:47)

Email + deliverable delivery legs for briefs
Effort: S · **Impact:** medium · **Feasibility:** verified

Two cheap delivery fan-outs after a run: (a) email — runner enqueues {template:'briefing', to: owner, subject, text: fullMarkdown-rendered} on the existing SQS EmailQueue; email-dispatch/SES/idempotency all already handle it; (b) deliverable — write fullMarkdown as a dated file into the deliverables corpus via internal/deliv so briefs show up in the Download Center / Android Files tab with the 180-day lifecycle. Both are recipe delivery flags from idea 1. Email leg makes the brief useful even on days the owner never opens a session.

Prerequisites: Idea 4

Risks: Almost none — both pipelines are prod-verified; respect SES From @jeremy.ninja / Reply-To gmail rules

Seen-item ledger: dedup + freshness cache so briefs only contain what's new
Effort: M · **Impact:** medium · **Feasibility:** verified

Per-recipe cache layer: SRCSEEN# item holding a bounded set of content hashes (feed GUIDs, GitHub notification ids, headline hashes) with 14d TTL, checked by fetch_feed/github_digest via a sinceToken the runner passes, so Tuesday's brief never re-reads Monday's stories. Also cache raw source responses at BRIEFCACHE# for ~15 min so an on-demand 'run my brief again' right after the scheduled run costs near-zero tokens and no re-fetch. Pure Query/GetItem single-partition, TTL-expired — matches every data rule.

Prerequisites: Ideas 4, 6

Risks: Hash-set item growth — cap at ~500 entries with oldest-first eviction; conditional-write races are harmless (worst case a dup item)

Commute briefing: Android auto-start on car Bluetooth connect
Effort: L · **Impact:** high · **Feasibility:** verified with corrections

The headline UX: a BroadcastReceiver for BluetoothDevice.ACTION_ACL_CONNECTED filtered to the owner-selected car device (picker in Android settings, stored in the versioned settings doc as briefing.carBtDeviceMac — additive field). On match within the commute window, start the existing session flow via RealtimeSessionCoordinator with an autoBrief intent: session opens, the idea-8 injected cue + read_briefing fires immediately, and the brief plays over the car's A2DP/HFP route. Wake word FGS is already running, so the receiver can hand off to it; falls back to a high-priority 'Briefing ready — tap to play' notification if Android 14/15 BT-launch FGS restrictions bite.

Prerequisites: Ideas 4, 8; Android has NEVER been run on a real device (known repo gap) — that verification gates this whole leg

Risks: Highest-risk idea: BLUETOOTH_CONNECT runtime permission, OEM background-start restrictions, mic-FGS-from-background rules vary by Android version; quota gate must treat the auto-session as a normal mint (it does). Ship notification-tap fallback first, true auto-play second

> **Verifier corrections:** Code anchors real: WakeWordService.kt (FGS), WakeBootReceiver.kt precedent for a manifest receiver, RealtimeSessionCoordinator.kt (+ its unit test), transport prime() (GeminiLiveTransport.kt:104), contracts/settings.schema.json is additionalProperties:true at every level so briefing.carBtDeviceMac is a legal additive

> field, and internal/sync (FCM fan-out) exists. The risks section is honest and correctly the longest: BLUETOOTH_CONNECT runtime permission, Android 14/15 background FGS-with-microphone start restrictions, OEM kill lists — and the repo-wide caveat that Android has not been verified on a real device gates everything. The stated mitigation order (notification-tap fallback first, true auto-play second) should be treated as mandatory, not optional. Quota claim correct — auto-session mints through the same CheckMint gate. Effort L honest.

Background-run cost gate: per-recipe budget in the quota system
Effort: S · **Impact:** high · **Feasibility:** verified

Scheduled agentic runs spend tokens with nobody watching, so extend realtime/quota.go with a CheckBriefing gate: reuse the existing suspension check + monthly-token cap, add QUOTA_BRIEFING_DAILY_RUNS (default ~6) and a per-run token ceiling; runner records actual usage into the same USAGE#month counter so briefs share the $15/month envelope and show up in /v1/costs month-to-date and the history cost column. On budget breach, skip the run and put one line in the next brief ('skipped 2 scheduled runs — monthly budget'). Follows the exact envFloat/envInt override pattern the gate already uses.

Prerequisites: Idea 4

Risks: None architectural — this is the owner's own dominant constraint (cost aversion) made enforceable; without it, idea 5 shouldn't ship

Web /briefings page: recipe editor + run history viewer
Effort: M · **Impact:** medium · **Feasibility:** verified

SSR page + ES-module orchestrator (the conversation.mjs pattern): list recipes with enable toggles, edit promptText/sources/schedule/delivery with every enumerable value as a populated control (UX-R01 — schedule as preset picker 'weekday mornings 6:30' not a cron text box; sources as typed add-source rows: RSS URL / GitHub / weather / crypto), a 'Run now' button, and past briefs rendered as structured section cards (never raw JSON) with cost badges priced from rates. Reuses the settings drawer's optimistic-PUT/409-reconcile loop for recipe saves. Voice editing (idea 2) remains the primary path; this is the inspect/repair surface.

Prerequisites: Ideas 1, 4

Risks: Low; mandatory multi-persona design pass (UX-R07) before the form ships

Market/prices source: keyless crypto + stocks quote tool
Effort: S · **Impact:** medium · **Feasibility:** verified

Direct answer to the owner's example utterance ('add crypto prices to my morning brief'): a get_market_prices tool with enumerated symbol params hitting keyless public APIs (CoinGecko simple/price for crypto; Stooq/Yahoo-style CSV endpoint for indices/tickers), returning {symbol, price, 24hChangePct}. Same keyless-external-API shape as weather.go (Open-Meteo) — small struct, structured parse, 5s timeout, no secrets. The recipe stores the symbol list; the refiner (idea 3) edits it by voice.

Prerequisites: None (usable standalone in live conversation immediately, before any briefing infra exists)

Risks: Free-tier endpoint stability — return a typed unavailable error the model can speak around; pick endpoints without auth or aggressive rate limits

Tab5 morning-brief ambient cue via device shadow

Effort: M · **Impact:** low · **Feasibility:** verified with corrections

Low-effort desk-surface tie-in: when a brief lands, the runner publishes a briefingReady flag + one-line headline into the Tab5 config shadow (or a liveninja//control/down message via the existing device_control publish path). ln_iot already consumes shadow deltas; ln_ui's Idle screen shows a subtle 'Briefing ready — say Hi Lily' chip. Saying the wake word starts a session where idea 8's injection offers the brief. No new firmware subsystems — one shadow field + one Idle-screen element, honoring the select-only/one-decision LCD rules.

Prerequisites: Ideas 4, 8; Tab5 pairing e2e (ProvisionIoT stub) remains the known hardware-gated blocker for real devices

Risks: Firmware change = HIL verification cycle; keep the shadow field additive per the 10-year contract horizon

> **Verifier corrections:** Firmware anchors real: contracts/shadow.md, cmd/shadow-ingest, LN_IOT_EVENT_CONFIG_DELTA in firmware/components/ln_iot (ln_iot.h, ln_iot_shadow.c), ln_ui idle screen (ln_scr_idle_ *in ln_ui.c). Correction on IAM: the web fn's iot:Publish grant (Sid DeviceControlPublish, template.yaml:285-291) covers topic/liveninja/* only — device shadow updates go over $aws/things//shadow/* topics, which that grant does NOT cover, so the shadow route needs iot:UpdateThingShadow (or a $aws topic grant) added; the control/down publish path avoids the IAM change but is fire-and-forget with no retained state for a device that's asleep, so shadow is the right mechanism and the grant addition should be planned. Also the publisher is the runner Lambda (idea 4), which needs its own iot grant + DescribeEndpoint. Effort M honest given the HIL firmware verification cycle; Tab5 pairing e2e gap correctly acknowledged. Impact 'low' is a fair self-rating.

Car activation, Bluetooth remote & commute UX (13 suggestions)

Direct answer: yes, a Bluetooth remote can activate it — with honest Android caveats. A `MediaSessionCompat` hosted in the existing wake-word foreground service captures AVRCP media-button presses (steering wheel or a cheap BT button); the catch is Android routes media buttons to the most-recently-active media session, so if Spotify is playing, Spotify wins — manageable via a dedicated button, car-connect re-assertion, and a settings toggle. The *most reliable* signal is Bluetooth car-connect itself (`ACTION_ACL_CONNECTED` filtered to your car MAC): auto-arm wake, flip car-mode defaults, offer the briefing. The biggest quality determinant nobody talks about: **HFP vs A2DP** — the car mic is narrowband SCO (degrades wake-word models and ASR), while phone-mic + car-speaker A2DP keeps wideband capture but breaks echo cancellation. An `AudioRouteController` with an explicit mode setting confronts this head-on (and the verifier found today's code hard-routes every session to the phone's built-in speaker, which would actively fight car Bluetooth — so this is load-bearing). Android Auto projection is an honest no-go (no assistant app category); coexistence hardening is the 1-2 day win instead.

MediaSession media-button capture: AVRCP play/pause as car PTT trigger

Effort: M · **Impact:** high · **Feasibility:** verified

Host a MediaSessionCompat (active, with a silent playback state) inside WakeWordService so a steering-wheel play/pause or a cheap BT media button fires an AssistTrigger — exactly the path wake detection already uses. Map single-press = start/PTT, double-press = end session, long-press (via KEYCODE_MEDIA_* discrimination) = interrupt/barge-in. HONEST CAVEAT: Android routes media buttons to the most-recently-active media session

— if Spotify is playing, the button goes to Spotify, not Live Ninja. This works reliably when (a) the user isn't playing other media, (b) a dedicated second BT button (not the steering wheel) is paired to the phone, or (c) the session is made active on car-connect (idea below) and re-asserted after each assistant turn. Ship it with an explicit 'media button = assistant' toggle in SettingsScreen so the owner can disable it when it fights with music apps.

Prerequisites: androidx.media dependency; decide press-pattern mapping; the session-survival FGS idea below makes double-press-to-end meaningful

Risks: Media-button routing ambiguity with music apps is inherent to Android — cannot be fully fixed, only managed; some cars send AVRCP only to the active A2DP source

Bluetooth car-connect auto-arm (the reliable non-wake-word activation signal)

Effort: M · **Impact:** high · **Feasibility:** verified

A manifest-registered receiver for BluetoothDevice.ACTION_ACL_CONNECTED/DISCONNECTED (still exempt from implicit-broadcast restrictions, unlike ACTION_POWER_CONNECTED which is NOT manifest-deliverable since API 26) filtered to the car's saved MAC address. On connect: start WakeWordService if enabled, activate the MediaSession (previous idea), flip car-mode defaults, and post a high-priority 'Tap to talk — driving?' notification with a full-screen-capable PendingIntent into MainActivity ACTION_ASSIST. On disconnect: tear it all down. This is the same protected-broadcast receiver pattern WakeBootReceiver already uses for BOOT_COMPLETED, including its Android-15 no-mic-FGS-from-background fallback: starting a mic FGS from a BT-connect broadcast is allowed only if the app is in an eligible state, so reuse WakeBootReceiver's tap-to-resume notification fallback verbatim. Add a one-time 'pick your car' device chooser (BLUETOOTH_CONNECT runtime permission) in settings.

Prerequisites: BLUETOOTH_CONNECT permission + chooser UI; verify mic-FGS-from-BT-broadcast eligibility on Android 15 (fallback path exists either way)

Risks: Some cars connect BT while owner is near but not driving (garage); mitigate with a disconnect-timeout and the notification (arm, don't auto-listen)

Audio-route manager: confront the HFP-vs-A2DP microphone reality explicitly

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

The single biggest car-quality determinant, currently unhandled (zero SCO/routing code in the app). Reality: (1) car mic requires HFP/SCO — 8kHz CVSD or 16kHz mSBC, narrowband/compressed; the openWakeWord ONNX models and EnergyVad are tuned for 16kHz phone-mic audio and will degrade, and OpenAI/Gemini ASR quality drops too; (2) phone mic + A2DP car-speaker output keeps wideband capture but adds 100-200ms A2DP latency, which breaks AEC correlation → the assistant hears itself → self-barge-in (the AEC-is-load-bearing constraint). Build an AudioRouteController used by WebRtcTransport/GeminiLiveTransport/NovaBridgeTransport: modes 'Car mic (HFP)' via AudioManager.setCommunicationDevice(BluetoothDevice) — note VOICE_COMMUNICATION source already biases toward SCO once a communication device is set — 'Phone mic + car speakers (A2DP)', and 'Phone only'. Default to HFP for calls-like reliability (cars are engineered for HFP echo paths), expose the choice in car-mode settings, and disable wake-word listening while SCO is active (feed it nothing rather than narrowband garbage). This needs real in-car testing on the owner's commute — budget for that, not just code.

Prerequisites: Physical in-car verification (the Android app has never run on a real device — that gate comes first); BLUETOOTH_CONNECT for device enumeration

Risks: SCO setup latency (~1s) delays session start; per-car AEC behavior varies wildly; A2DP mode may make patient-mode barge-in unusable

> **Verifier corrections:** CORRECTION: the claim 'zero SCO/routing code in the app' is false. All three transports already set a communication device — hardcoded to TYPE_BUILTIN_SPEAKER: WebRtcTransport.configureAudioForCall/restoreAudioMode at realtime/WebRtcTransport.kt:326-350, GeminiLiveTransport.kt:662-677, NovaBridgeTransport.kt:~440-450. This actually STRENGTHENS the idea: today every session force-routes to the phone's built-in speaker (MODE_IN_COMMUNICATION + setCommunicationDevice(BUILTIN_SPEAKER)), which will actively fight car BT audio. Also SettingsViewModel.enumerateMicDevices (:413-450) already enumerates TYPE_BLUETOOTH_SCO inputs and schema has micDeviceId — but nothing applies micDeviceId to transports. So the deliverable is refactoring three duplicated hardcoded routing blocks into one AudioRouteController honoring a mode setting, not greenfield routing. VOICE_COMMUNICATION source confirmed in all raw-audio transports (Gemini :582, Nova :370) and WebRTC (JavaAudioDeviceModule, HW AEC/NS at :316-319). transport.prime() seam exists (RealtimeSessionCoordinator.kt:101). Effort L honest (in-car verification is genuinely the long pole — the app has never run on hardware per plan.md).

Session foreground service: keep the conversation alive screen-off in the car

Effort: L · **Impact:** high · **Feasibility:** verified

Today the realtime session lives in ConversationViewModel/RealtimeSessionCoordinator scoped to the Activity — screen-off, app-switch, or process trim mid-commute kills the call (ConversationViewModel already tracks appInBackground but has nowhere to move the session). Promote the active session into a mic-type foreground service (or extend WakeWordService with a 'session held' mode since it already holds the mic FGS type and notification channel), owning the coordinator lifecycle, with notification actions End / Mute / Interrupt. This is the prerequisite that makes every other car idea real: phone in pocket or mount, screen off, conversation continues. Also the natural home for the MediaSession and AudioRouteController.

Prerequisites: None beyond the existing FOREGROUND_SERVICE_MICROPHONE permission; decide service-vs-WakeWordService-extension

Risks: Doze/OEM battery killers on long sessions; 10-min server session cap already bounds exposure

Car mode: hands-free-first defaults auto-applied when the car is detected

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

A client-side mode (triggered by BT-car-connected, optionally corroborated by ActivityRecognition IN_VEHICLE) that flips a coherent bundle of defaults for driving: hands-free continuous listening (no keep-warm timeout), wake word armed, mic eagerness high (interruptions are fine in a car, patient mode's 350ms confirm gate is wrong there), overlay bubble suppressed, screen interactions never required, and optionally the per-device engine pin flipped to gemini-flash-live for the daily hour of cheap audio (per-device voiceEngine pin already exists in settings — the phone can simply carry a second logical deviceId or a carMode override field, additive per contracts rules). Store as an additive carMode object in the settings doc so web can edit it too and unknown-field round-trip keeps old clients safe.

Prerequisites: BT car-connect detection landed first; schema addition follows contracts/README.md additive-only rules

Risks: ActivityRecognition adds Play-services dependency + permission; BT-connect alone is probably sufficient signal for a single-owner system

> **Verifier corrections:** Verified: micEagerness in contracts/settings.schema.json:232-241; per-device voiceEngine pin resolved server-side in internal/realtime/mint.go ResolveEngine :747-789 (devices[deviceId] ?? default, projected single read); contracts/README.md rules 1-2 are additive-only + unknown-field round-trip, and SettingsStore.update() preserves unknown fields (SettingsStore.kt:78-87). Three corrections: (1) 'patient mode's 350ms confirm gate' is not in the code — patient mode is micEagerness=low → semantic_vad eagerness low + interrupt_response:false (plan.md:639); the directional point (patient mode wrong for car) still stands. (2) 'the phone can simply carry a second logical deviceId' does not work — deviceId is bound to the auth session JWT (webapp middleware DeviceID(c)), not client-chosen per-mint; use the carMode-override field or a mint query param instead. (3) Biggest gap: Android has NO settings sync — SettingsStore is explicitly local-only ('Server push/pull sync is an M6 task', SettingsStore.kt:63-65) and LiveNinjaApi.kt has no settings PUT — so 'store carMode in the settings doc so web can edit it too' requires shipping M6 sync, or keeping carMode client-local and signaling the server via an additive mint query param. The client-side defaults bundle itself is fully feasible. Effort M fair with those scoping fixes.

One-tap 'Morning briefing' — shortcut that opens a session pre-seeded with a briefing intent

Effort: M · **Impact:** high · **Feasibility:** verified

A static app shortcut (long-press icon) + notification action on the car-connect notification: launches MainActivity with ACTION_ASSIST plus a new EXTRA_OPENING_PROMPT ('Give me my morning briefing'). After the transport reaches ready, the coordinator sends it as the first user text turn — the existing server tool catalog does the rest live (get_weather, recall_note, memory_search, plan_upsert'd plans, guides). Add a 'Briefing' guide or built-in persona server-side defining the briefing shape (weather, today's plan tasks, reminders, yesterday's open topics from /api/v1/topics) so it's one edit to tune. 'Resume' comes free: because topics-extract already tags conversations, the briefing guide can say 'if the user says continue, fetch recent topics via memory_search'. No new backend endpoints needed — this is intent-plumbing plus one guide.

Prerequisites: Session FGS idea makes this survivable screen-off; guide authored via existing /api/v1/guides

Risks: First-turn text before setupComplete on Gemini needs the same gating the transports already do for audio

Do-not-read-secrets-aloud: server-enforced car/voice discretion directive

Effort: M · **Impact:** medium · **Feasibility:** verified

When the mint request carries surface/car-mode context (a new additive query param or the deviceId's carMode setting), the broker appends a discretion directive to the composed instructions — same mechanism as accentDirectives and guide injection: 'Never read verbatim passwords, one-time codes, account numbers, or financial figures aloud; summarize and offer to send details via email instead.' Because instructions are server-composed and ephemeral tokens are config-bound, the client can't strip it. Pair with a belt-and-suspenders tool-router touch: in car mode, file_read and memory results containing high-entropy token patterns get a 'redacted-for-voice, use send_email' wrapper. The send_email tool (owner inbox default) is the natural safe channel and already exists.

Prerequisites: carMode settings field from the hands-free-defaults idea (or just an X-LN-style query flag)

Risks: LLM compliance with 'don't read X' is probabilistic — the tool-side redaction is the hard guarantee; keep both

Quick Settings tile + lockscreen-capable talk affordance
Effort: S · **Impact:** medium · **Feasibility:** verified

A TileService ('Talk to Ava') that fires the ACTION_ASSIST path — one swipe-and-tap from anywhere including (on many OEMs) the lockscreen QS shade, no assistant-role gesture needed. MainActivity already handles the over-keyguard case (setShowWhenLocked/setTurnScreenOn at :117-120) and KeyguardGate already gates sensitive tools while locked, so the security posture is done. Cheap, useful outside the car too, and a fallback for the days the BT button fights with Spotify. Add static shortcuts (Briefing / Talk / Mute wake) in the same pass.

Prerequisites: None

Risks: TileService.startActivityAndCollapse deprecation dance on API 34+ (PendingIntent variant) — minor

BLE HID / dedicated button support — scoped honestly
Effort: S · **Impact:** medium · **Feasibility:** verified

Cheap BLE buttons come in three flavors with very different Android realities: (1) buttons that present as HID consumer-control (media keys) — these Just Work via the MediaSession idea, zero extra code; recommend buying one of these (e.g. Tunai Button-class devices) and this is the primary recommendation; (2) camera-shutter buttons sending KEYCODE_VOLUME_UP — interceptable ONLY while an Activity is foreground (onKeyDown in MainActivity; useful as in-session PTT when the app is in a car mount, useless for background activation — an AccessibilityService could do it globally but is a Play-policy and privacy non-starter, do not build); (3) proprietary-GATT buttons (Flic et al) needing vendor SDKs — skip. Deliverable: foreground volume-key PTT in ConversationScreen + a docs note steering the owner to media-key-class hardware, plus CompanionDeviceManager pairing so the chosen button survives OEM BT power management.

Prerequisites: MediaSession idea for the background case; buy-the-right-button guidance is part of the deliverable

Risks: Volume-key capture steals volume control while in-app (gate it to car mode / an explicit toggle)

Android Auto: honest no-go on projection, small win on coexistence
Effort: M · **Impact:** medium · **Feasibility:** verified

Straight answer for the capability map: a custom voice assistant is NOT a buildable Android Auto app category — the Car App Library allowlist (media, messaging, navigation, POI, IoT) has no assistant slot, AA reserves the assistant surface for Google Assistant/Gemini, and Play's AA review would reject it. Do not spend time there. What IS worth 1-2 days: coexistence hardening so Live Ninja works as a phone app while AA is projecting — request AUDIOFOCUS_GAIN_TRANSIENT (ducking nav prompts), handle focus loss by pausing assistant TTS playback rather than killing the session, and verify SCO grab behavior while AA holds the car's media channel. Result: owner runs AA for maps, presses the BT button, talks to Ava over the car speakers, nav resumes after.

Prerequisites: Audio-route manager idea; real car with AA for verification

Risks: AA + assistant audio focus interplay is OEM-variant; timebox the verification

Car-noise wake profile: sensitivity + VAD retune bound to car mode

Effort: S · **Impact:** medium · **Feasibility:** verified with corrections

Road noise at 70mph shifts the energy floor, so EnergyVad's gate either never opens (wake dead) or always opens (ONNX burns battery and false-accepts rise). Add a car-mode wake profile: raised EnergyVad threshold with rolling noise-floor adaptation, slightly lowered oww head threshold (sensitivity mapping already exists in WakePreferences), and longer refractory. Since duty-cycling logic already reacts to Battery Saver/thermal via runtime receivers in WakeWordService, car mode is just one more input to that recompute. Small, but it's the difference between the wake word working in the car at all versus the owner concluding it's broken.

Prerequisites: carMode signal; ideally a few minutes of recorded in-car audio to tune against

Risks: Tuning without in-car samples is guesswork — pair with the device-verification drive

> **Verifier corrections:** Anchor correction: threshold + refractory live in wake/OpenWakeWordEngine.kt, not OwwPipeline.kt — REFRACTORY_MS=2500 at :62, refractoryUntil at :176/:193/:223, threshold=(1f-sensitivity).coerceIn(0.05,0.95) at :217 mapping off WakePreferences.sensitivityFlow (:59). OwwPipeline is the pure feature pipeline (refractory appears only in a comment :56). EnergyVad has a FIXED constructor threshold (200 RMS, EnergyVad.kt:22) with no noise-floor adaptation and is presumably constructed with defaults inside the engine — 'rolling noise-floor adaptation' is new algorithm code plus plumbing a carMode input into the engine, not just a constant swap. WakeWordService recompute is :203-219 (claimed :210-214, near). Effort S is slightly optimistic if the adaptive floor is included (S-M); without in-car audio samples the tuning-is-guesswork risk is correctly flagged. Concept sound.

Commute cost guard: auto-pin gemini-flash-live for car sessions

Effort: S · **Impact:** high · **Feasibility:** verified with corrections

A 1-hour daily commute at OpenAI Realtime rates will chew the ~$15/month ceiling (~30 min/day quota) fast; Gemini Flash Live is ~10x cheaper audio and already shipped (M13, pending E1/E2 live verification). When car mode is active, mint with the car deviceId (or carMode override) pinned to gemini-flash-live so commute minutes ride the cheap engine while desk sessions keep the default. Server support already exists end-to-end (per-device pin resolution, rates.go pricing, cost badges); this is client-side pin plumbing plus surfacing the engine name in the car notification so the owner knows which brain answered. Honest coupling: worthless until the owner completes Gemini E1/E2 verification — sequence it after.

Prerequisites: Gemini E1/E2 live-audio verification (owner-gated, already pending); carMode

Risks: Gemini has no server cancel primitive (barge-in is local flush only) and ~10-min goAway recycles — acceptable for commute chat, but set expectations

> **Verifier corrections:** Server side fully verified: ResolveEngine per-device pin at mint.go:747-789; rates.go exists; GeminiLiveTransport fully implemented; quota gate confirms the premise (~30 min/day + ~$15/mo pre-spend caps in quota.go); Gemini caveats confirmed in gemini-plan.md — no client→server cancel primitive (:99), ~10-min goAway recycles (:104), E1/E2 owner-gated (:178/:439/:563). CORRECTIONS: (1) cited anchor 'net/LiveNinjaApi.kt settings PUT' does not exist — LiveNinjaApi has putGuide only; Android settings sync is the unshipped M6 task and SettingsStore is local-only, so the phone cannot flip its own devices[deviceId] pin today (SettingsStore.setVoiceEngineDefault :102 is

> local, and the client doesn't even surface its own deviceId). (2) Practical paths: owner sets the phone's per-device pin once via WEB settings (works today via settings_routes.go but is static, not car-conditional), OR add an additive engine-preference query param to GET /api/v1/realtime/session honored only toward cheaper engines (small server change, quota-safe), OR ship the M6 sync. Effort S understates the car-conditional variant — M realistic. Sequencing after E1/E2 is honest and correct.

Geofence / driving-detection auto-arm — build only if BT-connect proves insufficient

Effort: M · **Impact:** low · **Feasibility:** verified

The alternatives to BT-connect, ranked honestly: (1) ActivityRecognition transition API (IN_VEHICLE enter/exit) — no location permission needed for the transition API itself on modern Android, moderate latency (30-120s), decent battery; (2) Geofencing around home/office departure — requires ACCESS_FINE_LOCATION + ACCESS_BACKGROUND_LOCATION (a heavy Play-review and privacy cost for a personal app that explicitly avoids third-party analytics), and 'left home' is a weaker driving signal than 'car BT connected'. Recommendation: ship BT-connect first (it is deterministic for this owner's one car), add ActivityRecognition as a corroborating signal only if the car's BT proves flaky, and skip geofencing entirely unless the owner wants departure-time briefing prefetch — which the server could do more cheaply as a weekday-morning one-shot via the existing EventBridge Scheduler email pattern anyway.

Prerequisites: Evidence that BT-connect misfires; Play-services ActivityRecognition dependency

Risks: Background-location permission is a disproportionate cost; recommend not building the geofence leg

Robustness & self-awareness (12 suggestions)

Your "highly robust, even self-aware" requirement becomes a concrete suite: a `self_status` tool (engine health, quota remaining, last deploy, error counts — spoken on request), an engine **health ledger with automatic fallback switching** at mint (three engines exist; today an OpenAI outage is dead air), a nightly self-test agent that exercises mint/tools/transcripts per engine and emails a pass/fail before your commute, a watchdog metrics sweeper (needs your sign-off — you removed CloudWatch alarms, and this is alarm-shaped even though it lives in code you control), deep provider canaries, client reconnect postmortems into the existing Athena lake, a weekly trend report, and a mint-response advisory so the system *tells you* when it self-healed ("heads up — running on the backup engine today").

self_status tool — the assistant reports its own health on request

Effort: M · **Impact:** high · **Feasibility:** verified

A new server-side tool ("how are you doing?" / "system status") that returns a spoken-friendly health snapshot: current engine pin and whether its last N mints succeeded, quota remaining (daily minutes, monthly tokens, concurrent-session slots), suspension state, last deploy sha+time, and recent error counts (MintErrors/FallbackErrors/QuotaRejections) pulled via cloudwatch:GetMetricData over the EMF-derived metrics that already exist. Register in internal/tools + mirror in the realtime toolManifest so voice sessions can call it; render as a structured card on web via the existing tool-card path.

Prerequisites: cloudwatch:GetMetricData IAM grant for the web fn in template.yaml (on-demand, no standing cost); a DEPLOY# record or embedded build-sha for 'last deploy' (see what_changed idea)

Risks: GetMetricData adds ~1-2s latency to the tool call — cache the snapshot in a short-TTL DynamoDB item; keep registry.go and mint.go manifests in sync (known drift trap, qa-report §2)

Engine health ledger + automatic fallback-engine switching at mint

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

The broker already emits MintErrors per engine; additionally write a rolling HEALTH# DynamoDB item (consecutive failures, last success ts) on every mint outcome. ResolveEngine then consults it: if the pinned engine has >=N consecutive mint failures in the last M minutes, fall through the healthy-engine order (openai-realtime -> gemini-flash-live -> openai-realtime-mini), return the substituted engine in the mint response with a `engineFallback` reason field (additive, never reusing wsUrl/bridgeUrl names), and let the client announce "OpenAI is down, switching to Gemini." This is the single biggest robustness win for the car commute — three engines exist and today an OpenAI outage just means dead air.

Prerequisites: Decide fallback order and per-persona voice mapping across engines (geminiVoice map already exists in realtime/personas.go); additive mint-response field per contracts/README.md rule 3

Risks: Mint success does not guarantee WSS/media-path success (mint-level health is a proxy — the deep canary idea closes that gap); flapping between engines mid-commute is worse than a clean failure, so require hysteresis; cost profile changes silently when falling back (surface it in the cost badge via rates.go)

> **Verifier corrections:** Anchors real: ResolveEngine at mint.go:747-808, validEngine :811, handleMint engine branches at cmd/realtime-broker/main.go:319-327 (exact), mirrored-SK isolate pattern confirmed (personas_store.go:45-49, quota.go:88-99), and the broker ALREADY has dynamodb:PutItem/UpdateItem on the whole table (template.yaml TableUsageBucketLog, ~:500-507) so HEALTH# writes need no IAM change. Two corrections: (1) the additive-field rule is contracts/README.md rule 1 (additive-only within /v1); rule 3 is enum growth. (2) The idea misses that falling from openai-realtime to gemini-flash-live changes the bootstrap MODE and shape (openai-direct vs gemini-direct, api_routes.go:481-526) — firmware ln_realtime maps only openai-realtime/mini to OPENAI_DIRECT (ln_rt_internal.h:19), so an automatic cross-engine substitution can hand a surface a bootstrap it cannot parse. Fallback must be surface/capability-gated; the safe universal first hop is openai-realtime -> openai-realtime-mini (identical shape), with gemini in the chain only for surfaces that negotiated gemini-direct support. Persona GeminiVoice mapping exists (personas.go GeminiVoice field + ResolveGeminiVoiceChain in gemini_mint.go). Effort L is honest.

Nightly self-test agent — exercises mint/tools/transcript and emails a morning report

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

A new scheduled Lambda (clone the UsageRollupFunction rate-schedule shape) running ~06:00 owner-local: mints a session for each enabled engine (mint-only validates keys, quota plumbing, persona resolution), invokes a benign tool round-trip through the registry (get_weather, recall_note, memory_search), posts and reads back a synthetic transcript, checks the email DLQ depth and last usage-rollup run, then enqueues a pass/fail report to the existing email queue. The owner reads one email over coffee and knows the whole stack is green before getting in the car.

Prerequisites: A canary identity: either a synthetic userId with its own quota partition or a quota-bypass path like the deliberately-ungated extract-topics mode (quota.go); canary transcripts need short TTL + exclusion from topics/history/costs so they don't pollute M11 data

Risks: Each OpenAI/Gemini mint consumes a real ephemeral token and broker slot (~60s hold, no release endpoint) — run serially and off-peak; false greens if mint-only (pair with the deep canary for media-path coverage)

> **Verifier corrections:** Anchors real: UsageRollupFunction with rate(1 hour) Schedule at template.yaml:604-631 (exact), EmailQueue + cmd/email-dispatch exist, POST transcript handler at api_routes.go:695, extract-topics is verbatim documented as deliberately NOT behind the quota gate (cmd/realtime-broker/main.go:603-608) — the cited precedent is genuine. Changes: the canary must direct-invoke the broker Lambda (same posture as the web fn, which has lambda:InvokeFunction on RealtimeBrokerFunction at template.yaml:284) rather than go through the HTTP surface, since API routes require a session JWT; the tools round-trip can run internal/tools in-process but Registry mandates a Reauthorize dep (registry.go:292-294), so the synthetic canary user needs a real PROFILE record that passes re-authorization; and 'exclusion from topics/history/costs' is not free — it requires explicit canary-session filtering in those readers, not just a short TTL. Mint-token burn concern is accurate (60s TTL, no release endpoint — RecordMint at main.go:373). Effort M is honest, trending M+.

Watchdog metrics sweeper — thresholded email alerts without CloudWatch alarms

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

The owner explicitly removed all CloudWatch alarms, so build the sanctioned substitute: a small scheduled Lambda (every 30-60 min) that runs GetMetricData over the existing EMF namespaces (LiveNinja/Realtime MintErrors+QuotaRejections, FallbackErrors, EmailDispatch, Deliverables ZipJobs error outcome, TopicExtractionErrors, ShadowIngest) plus GetQueueAttributes on the email DLQ, compares against thresholds, and enqueues an email ONLY on breach with a quiet-period dedupe (WATCHDOG# DynamoDB item so one incident = one email). Costs cents, zero standing infra, honors the no-alarms decision because notification logic lives in code the owner controls.

Prerequisites: cloudwatch:GetMetricData + sqs:GetQueueAttributes IAM in template.yaml; confirm with owner that scheduled-Lambda emails are inside the spirit of the no-CloudWatch-alarms decision (they are alarm-shaped)

Risks: Threshold tuning on a single-user system with bursty usage — start with error-only metrics (MintErrors, DLQ>0) where any nonzero value is signal; each run is 2 Lambda invocations/hour, negligible cost

> **Verifier corrections:** Technically clean and all anchors verified exactly: observ.go EmitMetric documented at :110-118 as claimed (EMF-only, no PutMetricData), EmailQueue + EmailDeadLetterQueue at template.yaml:1326-1340, schedule pattern :604-631. Metric names cited all exist (MintErrors, QuotaRejections, TopicExtractionErrors, TelemetryRecords, etc.). The required change is governance, not code: the owner decision recorded at plan.md:415 and :631 is 'NO CloudWatch alerts wanted' (alarms removed; budgets email directly), and this Lambda is functionally an alarm with a different implementation. Owner sign-off must be a BLOCKING prerequisite before any build, not a soft confirm — otherwise this idea is a 'no'. If approved, the error-only-thresholds start (any nonzero MintErrors, DLQ depth > 0) is right for a single-user system, and cost is negligible with zero standing infra.

what_changed tool + DEPLOY# ledger — the assistant can say what shipped

Effort: S · **Impact:** medium · **Feasibility:** verified with corrections

Add a step to deploy.yml that, after a successful deploy, writes a DEPLOY# record to the live-ninja table (sha, timestamp, commit subjects since previous deploy via git log, changed top-level areas from the paths-filter

outputs already computed in the workflow). A new what_changed tool reads the last N records and answers "what changed recently?" / "when did you last update?" in the car. Also gives self_status its 'last deploy' line and turns every push-to-main-is-prod deploy into an auditable spoken history.

Prerequisites: dynamodb:PutItem on a SYSTEM#DEPLOY partition added to the gha-deploy role; sanitize commit subjects (they become model-visible instruction-adjacent text — treat as data, truncate, strip anything odd)

Risks: Commit messages sometimes reference secrets-adjacent workflow details; keep to subject lines only and cap length

> **Verifier corrections:** Two factual corrections. (1) The paths-filter claim is overstated: the filter block is deploy.yml:56-68 (not :43-233) and classifies only TWO categories (containers/wakeword-train and nova) — it does not 'classify what changed per deploy'; the workflow would need its own git-log + area-classification step (easy, but new work, not reuse). (2) IAM: gha-deploy is an ORG-WIDE role managed outside this repo (credential-rotation/infra/github-oidc-deploy.yaml per deploy.md:51-52); it may already hold table write via its deploy permissions, but if not, the dynamodb:PutItem grant is an out-of-repo change — and scoping a shared org role to one repo's table partition is awkward. Cheaper alternative preserving effort S: skip the workflow write entirely — the deployed binary already knows its sha (version.go BuildVersion / X-LN-Server), so the web fn can self-record a DEPLOY# item on first request after a version change (existing table IAM, no role change), with commit subjects fetched-free (sha-only) or added later. Commit-subject sanitization concern is legitimate (model-visible text). Tool itself is a standard registry + manifest addition with a single-partition Query (no Scan) — fine.

Quota/cost 'fuel gauge' — spoken and visible remaining-budget awareness

Effort: S · **Impact:** medium · **Feasibility:** verified

A quick-win subset of self_status shippable independently: expose the quota gate's own numbers (daily seconds used/remaining, monthly tokens vs cap, month-to-date dollar estimate from /v1/costs, sessions in flight) as (a) a lightweight quota_status tool for voice ("how much talk time do I have left today?") and (b) a remaining-minutes strip next to the existing cost badge in conversation.mjs. All reads are single GetItems on items the gate already maintains — no new instrumentation.

Prerequisites: None beyond a tool registration + manifest entry; caps are env-derived so return them from the server rather than hardcoding client-side

Risks: Minimal; keep the response shape additive and remember the broker reads quota items via mirrored SK constants, not internal/store

Client session postmortem — self-healing reconnect audit via existing telemetry lake

Effort: M · **Impact:** medium · **Feasibility:** verified

realtime.mjs, the Android transports, and ln_realtime all reconnect/degrade silently today (Gemini goAway resumption, WSS backoff, connectionlost). Instrument each with a structured end-of-session postmortem event — engine, duration, disconnect cause taxonomy (goAway, ICE fail, token expiry, network, barge-in anomaly), reconnect attempts/successes, time-to-reconnect — POSTed to the existing telemetry ingest and landing in the Firehose->S3->Athena lake. A weekly Athena query (1GB scan cap already enforced) rolls it into a reliability summary. This is the data that tells the owner whether the Bluetooth-car-commute path actually holds up, and feeds honest numbers to self_status.

Prerequisites: Define the event schema (add names to the telemetry allowlist so they don't count as UnknownEventName); firmware leg rides the existing ln_iot telemetry path instead of HTTP

Risks: Three client codebases to touch — ship web first (highest iteration speed), Android/firmware follow; keep payloads PII-free per the no-PII-in-logs posture

Android wake-service health beacon + device last-seen in self_status

Effort: M · **Impact:** medium · **Feasibility:** verified with corrections

The Tab5 already heartbeats via ln_iot -> iot-ingest -> DEVICE#/TELEM records; Android has no equivalent, so a dead WakeWordService (killed by OEM battery manager, thermal duty-cycle stuck, boot-restart fallback pending) is discovered only when 'Hey Ninja' fails in the car. Add a low-frequency beacon from WakeWordService (every ~30 min while running, plus state transitions: muted, duty-cycled, model version, battery-saver state) through the telemetry route into DEVICE# items, and surface per-device last-seen + wake-state in self_status and the devices list. "Your phone's wake word listener hasn't checked in for 3 hours" is exactly the self-awareness the owner asked for.

Prerequisites: Android surface should reuse the HTTP telemetry route (it has no IoT identity); beacon must respect the contractual battery discipline — piggyback on existing wakeups, no new wakelocks or alarms

Risks: Staleness detection needs a reader-side threshold, not a server watchdog, to stay zero-standing-cost (evaluate last-seen at self_status/list time); Android app has still never run on a real device (backlog), so this lands with that workstream

> **Verifier corrections:** Core correction: the HTTP telemetry route lands events in the Firehose->S3 LAKE, not in DynamoDB — DEVICE#/TELEM last-seen items are written only by the MQTT iot-ingest path (cmd/iot-ingest/main.go:6, TelemetryRecords at :53, exact). So 'through the telemetry route into DEVICE# items' as written does not work: reading last-seen at self_status time from the lake means an Athena query per tool call (slow, costs). Fix: add a small DynamoDB side-write — either a designated beacon event name the telemetry handler additionally upserts to DEVICE#/TELEM, or a tiny authenticated /v1/device-beacon endpoint doing one UpdateItem. Reader-side staleness thresholds (evaluated at self_status/devices-list time) is the right zero-standing-cost design, as stated. WakeWordService.kt, WakeBootReceiver, duty-cycle machinery all exist in android/app/src/main/java/ninja/jeremy/liveninja/wake/. Honest caveat confirmed: the Android app has never run on a real device (plan.md:631), so this only proves out with that workstream. Effort M is honest including the server-side write path.

Post-deploy smoke gate in the deploy workflow

Effort: S · **Impact:** high · **Feasibility:** verified with corrections

Since push-to-main IS the production deploy with no staging, add a smoke job after the deploy step in deploy.yml: hit /healthz and /v1/compat, verify JWKS at /.well-known/jwks.json, and (with a CI-scoped credential path or broker direct-invoke via the OIDC role) perform a dry-run mint per enabled engine plus one read-only tool invoke. Failure marks the workflow run red and enqueues an email — catching a broken deploy minutes after push instead of at next morning's commute. Complements the nightly canary (which catches provider-side drift between deploys).

Prerequisites: A broker 'validate' sub-mode or reuse of mint with immediate slot awareness; lambda:InvokeFunction on the broker added to the gha-deploy role

Risks: A flaky provider (OpenAI blip) reddens a good deploy — make provider-mint legs warn-only and only infra legs (healthz/compat/JWKS) blocking

> **Verifier corrections:** Partially exists already: deploy.yml:436-448 has a retrying /healthz smoke step that is explicitly TOLERATED (non-blocking, pre-DNS-propagation rationale) — the infra leg of this idea is 'harden the existing step' (make it blocking post-first-deploy, add /v1/compat and /.well-known/jwks.json, both on the authorizer public allowlist, cmd/authorizer/main.go:61-63). That part is genuinely S and high-value. The mint leg needs two changes: (1) lambda:InvokeFunction on live-ninja-realtime-broker for the gha-deploy role is likely an out-of-repo change (org-wide role, credential-rotation repo per deploy.md:51-52) unless its deploy perms already cover it — verify first; (2) NEVER run a real session-mint from CI — the response carries a live client secret and per portable rules secrets must not enter CI logs/context; require a broker 'validate' sub-mode (new mode via the main.go:269-282 switch) that exercises key fetch + provider auth but returns pass/fail only. Warn-only provider legs vs blocking infra legs is the right split. With the validate mode, effort is S-M rather than S.

Deep provider canary — real WSS text-turn probe per engine

Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

Mint-level health misses media-path failures (token minted fine, WSS handshake or model turn fails). Add a broker mode 'canary' that opens the actual provider connection server-side — Gemini BidiGenerateContent WSS with an ephemeral token, OpenAI via a one-shot chat-completions/realtime text turn — sends a trivial text prompt, verifies a response arrives within a deadline, records latency to a HEALTH# item and an EngineCanary EMF metric. Invoked by the nightly self-test and, when the engine-fallback ledger shows a pinned engine failing, on-demand before switching — turning fallback decisions from guesses into verified facts.

Prerequisites: Engine health ledger idea (shared HEALTH# schema); Lambda outbound WSS to generativelanguage.googleapis.com (no VPC constraints on the broker, so fine); tiny per-probe token cost — meter it under a system identity

Risks: The broker is deliberately the sole key-holder and currently thin — keep canary code from bloating its cold start; WSS-from-Lambda needs a timeout well under the invoke deadline; probes cost real (tiny) provider dollars

> **Verifier corrections:** Anchor correction: the Request.Mode switch is at cmd/realtime-broker/main.go:269-282 as cited, but there is NO 'documented NEW BROKER MODE extension point' comment — that phrasing is invented. Adding a mode is nonetheless mechanical (mode string constant, switch arm, badRequest list, plus the webapp brokerRequest passthrough if ever web-triggered). Supporting machinery verified: realtime/gemini_mint.go ephemeral-token mint exists, realtime/fallback.go OpenAI HTTP client exists, gorilla/websocket v1.5.3 is already in go.mod (indirect), broker has no VPC so outbound WSS to generativelanguage.googleapis.com is fine. Depends on the HEALTH# schema from the engine-fallback idea (broker already has table write IAM). The stated risks are the real ones: keep the WSS deadline well under the invoke timeout, keep cold-start bloat down in the key-isolate Lambda, and meter probe spend under a system identity so the tiny real provider cost stays visible in the cost posture. Effort L is honest — this is the most speculative of the set; ship it after the mint-level ledger proves insufficient.

Weekly self-report email — reliability, cost, and usage digest

Effort: M · **Impact:** medium · **Feasibility:** verified

A Sunday-evening scheduled Lambda that composes one narrative email: sessions and minutes per engine, month-to-date cost vs the ~$15 ceiling (from the persisted per-session costs and rates.go), quota rejections, error counts by namespace, reconnect stats from the postmortem lake (one capped Athena query), wake-word trainings used, deploys that week (DEPLOY# ledger), and any watchdog incidents. Distinct from the nightly pass/fail canary: this is trend-level self-awareness — "Gemini saved you $4.10 this month; 3 sessions dropped mid-drive on Tuesday."

Prerequisites: Athena StartQueryExecution IAM for the new fn (results bucket already exists); graceful sections-degrade when a sibling ledger doesn't exist yet

Risks: Scope creep into a dashboard — keep it one templated text email; Athena query cost is bounded by the existing scan cap

Degraded-mode advisory — mint response carries a health hint the client announces
Effort: S · **Impact:** medium · **Feasibility:** verified

Small additive field on the mint response (e.g. `advisory: {level, message}`) populated by the broker from the HEALTH# ledger / recent MintErrors: engine substituted, provider slow, quota nearly exhausted, fallback-only mode. conversation.mjs shows a banner and, for voice-first surfaces (Android in car, Tab5), the client speaks one short line at session start ("heads up — running on the backup engine today"). Closes the loop so robustness work is *perceptible*: the system doesn't just self-heal, it tells you it did.

Prerequisites: Engine health ledger (or at minimum the quota gate's own numbers, available today); strictly additive field naming per the 10-year device contract — never wsUrl/bridgeUrl-family names

Risks: Three clients parse the mint response; unknown-field round-trip rules protect old clients, but the spoken advisory needs per-surface taste (must not delay time-to-first-word in the car)

Cross-domain & free-roam (17 suggestions)

Everything the focused themes do not cover, and several of the highest-leverage items live here: Google Calendar + Gmail triage (calendar is already sanctioned PRD phase-2), voice routines/macros ("run my leaving-work routine" fires N side effects through the existing per-step re-auth/idempotency pipeline), cross-surface conversation continuity (park the car, pick up on the Tab5), fire-and-forget research agents that email you a cited doc, deliverable dictation (memo by voice, waiting at your desk), spaced-repetition voice flashcards for the commute, memory-curation review ("what did you learn about me this week?"), a spoken cost fuel-gauge, and speculative-but-cheap bridges (Home Assistant over the existing IoT plane, household shared lists, email-in via SES receiving).

Google Calendar read/write tool (create_calendar_event + agenda queries)
Effort: M · **Impact:** high · **Feasibility:** verified

Add calendar tools to the server-side router: get_agenda (today/tomorrow/date range), create_calendar_event, and move/cancel. OAuth refresh token for the owner's Google account stored as SSM SecureString via the existing set-secret flow; a narrow Deps interface does the token refresh + Calendar API calls. Voice flows: 'what's

my day look like', 'push my 2pm 30 minutes', 'block Friday morning for deep work' — all from the car. This is already PRD phase-2 (create_calendar_event), so it's sanctioned direction, not scope creep.

Prerequisites: Owner completes a one-time Google OAuth consent to mint a refresh token (agent never sees it; set via set-secret).

Risks: Google OAuth refresh tokens for unverified apps expire in 7 days unless the OAuth app is in production mode — needs a real (personal) GCP project setup; token revocation handling.

Gmail voice triage: read, summarize, and draft replies to real inbox

Effort: L · **Impact:** high · **Feasibility:** verified

gmail_list_unread / gmail_read / gmail_draft_reply tools using the same Google OAuth credential as calendar. In the car: 'anything important in email?' → model summarizes top threads; 'draft a reply saying I'll review tonight' → creates a Gmail draft (never auto-sends from gmail — DMARC rule and safety both preserved; drafts only). Complements, not replaces, the existing SES send_email which stays the assistant's own outbound identity.

Prerequisites: Calendar-idea OAuth plumbing (same credential, gmail.readonly + gmail.compose scopes).

Risks: Inbox content is high-value PII flowing through the voice provider — should be an explicit per-settings opt-in; summarizing long threads eats fallback-model tokens (quota-gate the tool).

Reminders that actually reach the car: FCM/IoT delivery targets for set_reminder

Effort: M · **Impact:** high · **Feasibility:** verified with corrections

Today set_timer/set_reminder can only land as email (SchedulerRole's sole permission is sqs:SendMessage to the email queue). Add a delivery target enum {email, phone, tab5} to the reminder payload: extend SchedulerRole to invoke a small notify Lambda that fans out FCM data-push to Android (spoken aloud by the app if a session is live, notification otherwise) and IoT publish to liveninja//control/down for Tab5. 'Remind me to call mom at 5' finally interrupts you at 5pm in the car instead of sitting in an inbox.

Prerequisites: SchedulerRole widened with lambda:InvokeFunction (capability map flags this exact extension point); Android handler for a new FCM message type.

Risks: Additive contract change to the reminder payload; Android background delivery reliability (Doze) — keep email as guaranteed fallback.

> **Verifier corrections:** SchedulerRole confirmed at template.yaml:1879-1904 with sqs:SendMessage to EmailQueue as its SOLE permission (claim accurate); web fn iot:Publish on liveninja/* at template.yaml:285-290 confirmed; tools/scheduler.go targets the email queue via emailQueueMessage. TWO CORRECTIONS: (1) 'Android FCM settings fan-out already exists' is FALSE — there is no Firebase/FCM anywhere in the repo (no firebase dependency in android/, no messaging service, no token registration route; settings fan-out is IoT shadow to Tab5 plus poll-based reconcile, settings_routes.go:90,157). The Android leg means adding Firebase project + client SDK + FCM token registry route + storage from scratch. (2) The cited 'capability map' document flagging this extension point was not found in the repo. Tab5-IoT leg + notify Lambda alone is M; with the FCM leg it's L. Reminder payload is internal (scheduler->queue), so the additive-contract concern is mild; email fallback as guaranteed path is the right call.

Deliverable dictation: 'draft me a memo' → structured document by voice
Effort: M · **Impact:** high · **Feasibility:** verified

A compose_document tool + broker mode 'compose': user dictates intent and rough content during the commute; the broker runs gpt-4o-mini with a document-type-specific structured prompt (memo, PRD section, decision doc, email draft) and writes the result via the existing deliverable_create path, then deliverable_deliver emails it so it's waiting at the desk. Mirrors the proven extract-topics broker-mode pattern exactly (async invoke, strict-JSON, sanitized).

Prerequisites: None — all primitives exist.

Risks: Long dictations need the transcript, not just the tool-call args — cleanest as a post-session enrichment triggered by a tool flag, like topics-extract.

Cross-surface conversation continuity: park the car, pick up on Tab5/web
Effort: M · **Impact:** high · **Feasibility:** verified

On session final, write a small HANDOFF# item (last-N-turns summary + open threads, generated by the already-running topics-extract call — one extra JSON field). At next mint, if a recent handoff exists (<2h) inject a 'previous conversation context' block into instructions the same way guides are injected. 'As I was saying in the car...' just works on any surface. Optionally a resume_conversation tool to pull older sessions by topic.

Prerequisites: None.

Risks: Instruction-size budget (guides already cap at 6000 chars) — handoff block needs its own tight cap; staleness rules need care so mornings don't open with last night's context.

Spaced-repetition commute learning (voice flashcards)
Effort: M · **Impact:** high · **Feasibility:** verified

CARD# items with SM-2 scheduling fields under the user partition; tools card_add ('make a flashcard: ...'), card_review_next, card_grade. A 'Tutor' built-in persona runs a review session: asks due cards aloud, grades the spoken answer, updates intervals. Turns the 1-hour commute into deliberate learning with zero new infra — it's just single-partition Query on due-date and the existing tool loop. Genuinely novel use of the architecture and nearly free.

Prerequisites: None.

Risks: Voice-grading leniency (model judges 'close enough') — acceptable for personal use; card authoring by voice needs a quick web view later for editing typos.

Speak the money: get_usage_and_costs tool
Effort: S · **Impact:** medium · **Feasibility:** verified

Expose the data already behind /v1/costs and the usage-rollup (month-to-date spend, per-engine split, remaining daily minutes / monthly token budget from the quota gate) as a read-only tool so 'how much have I spent this month?' and 'how much talk time do I have left today?' work by voice. Also lets the assistant

proactively say 'heads up, you're at 80% of the monthly cap' when asked about status. Pure read of existing USAGE#/CONV# records.

Prerequisites: None.

Risks: Almost none — read-only, owner-only data.

Contact-aware email: 'send it to Sarah' via the entity graph
Effort: S · **Impact:** medium · **Feasibility:** verified

Add an email field (and phone, birthday) to person entities in the memory graph; teach send_email to resolve a recipient by person name through entity_get before the existing allowlist check (which stays mandatory — this changes addressing UX, not the exfiltration guard). 'Email Sarah the summary' stops requiring a spoken email address, which is miserable in a car. Also enables 'when is Dave's birthday?' from the same records.

Prerequisites: None.

Risks: Name-collision resolution needs a confirm turn ('which Sarah?'); keep allowlist as the hard gate so a hallucinated address can't send.

Voice routines/macros: 'run my leaving-work routine'
Effort: M · **Impact:** high · **Feasibility:** verified

ROUTINE# documents (name + ordered list of pre-bound tool calls with frozen args, e.g. send_email ETA to spouse → set_reminder for gym → device_control tab5 announce) created/edited by voice via routine_upsert and executed by run_routine, which invokes registered tools in-process through the same registry (so re-authz, idempotency, and audit all apply per step). One utterance triggers N side effects — the highest-leverage single feature for a repetitive daily commute.

Prerequisites: None.

Risks: Composite side effects need per-step failure semantics (continue vs abort) and a dry-run readback ('routine has 3 steps, run it?'); cap steps to keep it bounded.

Email-in: give the assistant an inbox (SES receiving → notes/tasks/memory)
Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

Enable SES inbound receipt on assistant@jeremy.ninja → S3 → Lambda that parses the mail and files it: forwarded articles become notes/deliverables, 'todo:' subjects become plan tasks, contact info updates the entity graph — then a confirmation email back. Closes the loop from the desk side: anything you can forward becomes queryable by voice on the next commute. Uses SES the stack already owns, just the receiving half.

Prerequisites: SES receipt rule + MX record on a subdomain (Route53 already in stack); strict sender allowlist = owner's addresses only.

Risks: Inbound email is an injection surface — treat body as untrusted data, never as instructions; sender spoofing mitigated by SPF/DKIM verdict checks in the receipt Lambda.

> **Verifier corrections:** Anchor corrections: template.yaml:633-679 is the EmailDispatchFunction (SES SEND policy referencing the jeremy.ninja identity ARN); SesConfigurationSet is actually at :2242-2255, and there is NO AWS::SES::EmailIdentity resource in the template — the identity is verified out-of-band, so 'the stack already owns SES' overstates the IaC footprint slightly. Route53 is in-stack (RecordSets at :2162/:2173, hosted-zone param :18) so the MX record is easy. Real gotcha to add: SES receiving uses an account/region-global ACTIVE ReceiptRuleSet — activating one is an account-wide switch and must be confirmed unused; receiving is also region-limited (us-east-1 OK). New S3 bucket + Lambda are $0-idle serverless — cost posture intact. The injection-surface and SPF/DKIM-verdict mitigations are correctly identified and essential. Effort L honest.

Meeting prep packs: entity graph × calendar × topics

Effort: L · **Impact:** high · **Feasibility:** verified with corrections

A prep_meeting tool (and/or automatic pre-commute run once calendar lands): for the next meeting, pull attendee person-entities from the memory graph, related topics/conversations from GSI3, open plan tasks mentioning them, and recent notes — assemble into a spoken 60-second brief plus a deliverable doc. This is the payoff feature that justifies calendar + memory existing in one system.

Prerequisites: Calendar integration shipped; entity graph populated enough to be useful (memory_write already accumulates).

Risks: Quality depends on memory hygiene; a thin graph makes thin briefs — ship after a few weeks of memory accumulation.

> **Verifier corrections:** ANCHOR ERROR: there is no GSI3 — the table has only GSI1/GSI2 (template.yaml:956-965), and store/topics.go:5 says 'deliberately NO new GSIs': topic/conversation history is single-partition sk-prefix Query (TREF##.. and CONV#.., topics.go:18-19). The data is all reachable, just via those Queries — do NOT add a GSI for this. memory_search cosine ranking confirmed (memory/search.go:64, cosine.go). Correctly gated on the calendar idea shipping and on graph density; deliverable_create confirmed. Effort L honest given the prerequisite. Fix the buildsOn wording and it's sound.

Evening debrief persona → auto-journaling and memory capture

Effort: S · **Impact:** medium · **Feasibility:** verified

A 'Debrief' built-in persona whose style block runs a structured end-of-day interview on the drive home (wins, blockers, people, decisions, tomorrow's top-3), with instructions to call memory_write/plan_upsert as facts emerge and remember_note for the journal entry; post-session, the compose_document pipeline (dictation idea) renders a dated journal deliverable. Pure prompt + existing tools — the cheapest high-value idea here.

Prerequisites: None (journal-render polish benefits from the compose_document idea).

Risks: None technically; persona needs the mandated multi-persona design pass (UX-R07) for its style block.

Household shared lists (shopping/errands) within the allowlist model

Effort: M · **Impact:** medium · **Feasibility:** verified

A HOUSEHOLD# shared partition holding named lists (shopping, errands, house projects) with list_add/list_get/list_check tools readable/writable by any allowlisted user — spouse adds 'milk' from web, owner

hears the list read out at the store. Deliberately NOT multi-tenant (PRD non-goal respected): it's the existing owner+allowlist household, one shared partition, owner-controlled. Speculative on whether other household members will actually adopt the app.

Prerequisites: A second household user actually signing in (allowlist mechanics already exist).

Risks: Edges the single-owner assumption baked into some queries (per-user partitions) — shared partition must get its own authz check; mark SPECULATIVE pending real second-user demand.

Fire-and-forget async research agent: 'look into X, email me the doc'

Effort: L · **Impact:** high · **Feasibility:** verified

A research_task tool that accepts a question, immediately returns 'on it', and async-invokes a worker Lambda (topics-extract shape) that runs a multi-step loop — web_research/web_lookup legs + broker fallback-turn synthesis — then writes a cited deliverable and emails it. Distinct from live web_research (which blocks the conversation): this is delegation, the assistant working while you keep driving. The 15-min Lambda ceiling suffices; the existing Batch queue is the escape hatch for bigger jobs.

Prerequisites: None; a per-day task quota knob via the QUOTA_* env pattern (quota.go NewGate) to bound spend.

Risks: Unbounded LLM/token spend if the loop isn't capped (hard iteration + token ceilings required); SSRF allowlist currently limits direct-fetch domains — research quality leans on HN/Wikipedia legs unless the allowlist grows deliberately.

Home-automation bridge: Home Assistant behind the existing IoT plane

Effort: L · **Impact:** medium · **Feasibility:** verified

SPECULATIVE (depends on owner running Home Assistant or similar). Extend device_control's pattern with a home_control tool: web fn publishes MQTT to a liveninja/home/control topic; a tiny local bridge (HA add-on or systemd service using an IoT thing identity, same fleet-provisioning path as Tab5) subscribes and relays to Home Assistant's local API. 'Turn on the porch lights, I'm 10 minutes out' from the car, with zero new AWS infra — IoT Core rules, policies, and the device identity model already exist.

Prerequisites: A home-automation hub exists at home; a small always-on process on it (local, not AWS — cost posture intact).

Risks: Action enum must stay fixed/strict like device_control (no free-form commands to home devices); local bridge is a new lifecycle to maintain.

Location-aware context and arrival triggers

Effort: L · **Impact:** medium · **Feasibility:** verified with corrections

SPECULATIVE. Android app registers coarse geofences (home/work, user-defined) and POSTs zone-transition telemetry to the existing telemetry route; latest zone lands as a CTX# item read at mint so the assistant knows 'driving, near work' and phrases answers accordingly; 'remind me when I get home' becomes a geofence-armed local reminder on the phone (privacy-preserving: zones only, no coordinates server-side, evaluation on-device). Complements the FCM-reminder idea.

Prerequisites: FCM/local-notification reminder work (shares delivery machinery); owner comfort with zone tracking (opt-in setting, aligns with privacy posture only if zones-not-coords).

Risks: Battery discipline is contractual on Android — must use platform geofencing, no polling; privacy review against the no-PII-in-logs rule.

> **Verifier corrections:** Anchors partially confirmed: telemetry_routes.go is Firehose Direct PUT only — adding a CTX# DDB write is new (simple, additive, but the route currently has no DDB dependency at all); settings.schema.json is additive-friendly (additionalProperties:true throughout); WakeWordService.kt FGS exists for the geofencing client. Two corrections: (1) contracts/telemetry.schema.json would need an additive zone-event type (contract change, allowed but must be done deliberately); (2) 'geofence-armed local reminder on the phone' presumes Android notification machinery that does not exist — no notification/push code in the app today, and the FCM prerequisite idea itself starts from zero Firebase. The zones-not-coords privacy design is sound. Effort L is honest for the context leg; the arrival-trigger leg inherits the reminders idea's understated Android work. SPECULATIVE flag appropriate.

Voice memory-curation review: 'what did you learn about me this week?'

Effort: S · **Impact:** medium · **Feasibility:** verified with corrections

A memory_review tool that Queries recent EMB#/ENT# writes (already time-prefixed) and has the model read back this week's new memories in batches — user says keep / fix / forget per item, driving memory_write corrections and the existing forget tool. Fixes the silent-drift problem every long-lived memory system develops, entirely with existing primitives plus one bounded Query; pairs naturally with the debrief commute slot.

Prerequisites: None.

Risks: Read-back of many items burns session minutes — batch to ~10 per pass and honor the daily quota.

> **Verifier corrections:** ANCHOR ERROR: EMB#/ENT# ids are NOT time-prefixed — entityID is uuid.NewString() (memory/write.go:56), giving sk=ENT## with no time ordering (the time-prefixed-id idiom belongs to NOTE# items in notes.go). However, Entity carries an UpdatedAt RFC3339 attribute (entities.go:85), so 'this week's writes' = bounded ENT# prefix Query (the partition is small, single-user) + client-side sort/filter on updatedAt — still no Scan, no GSI. forget (registry.go:322) and memory_write confirmed. With that query approach corrected, the idea works as described; effort S still roughly honest (S/M). Batching read-back to ~10 items per pass against the daily quota is the right mitigation.

Appendix: cross-cutting constraints the verifiers enforced

- **Cost posture:** no standing infrastructure — every suggestion above is $0-idle (Lambda, Scheduler, Batch-per-job, on-demand DynamoDB). The one idea that puts media bytes on AWS (podcast prefetch) is opt-in and flagged.
- **Key isolation:** provider keys stay in the broker; new credentials (GitHub PAT, Google OAuth) get their own scoped SSM params + IAM Sids, set via `scripts/set-secret` — agents never see values.
- **Contracts:** additive-only; new tools must be registered in BOTH `internal/tools/registry.go` and the hand-mirrored `toolManifest` in `internal/realtime/mint.go` (a documented drift trap) — Gemini and fallback derive automatically.

- **Data discipline:** single-table, single-partition Query only — no Scans, no new GSIs (two ideas were corrected for inventing a GSI3).
- **Security posture:** every side-effecting voice tool follows the `send_email` precedent — server-side allowlist AND spoken confirmation; the car adds a do-not-read-secrets-aloud directive with deterministic tool-side redaction.

Generated by a 16-agent review workflow (4 readers, 6 ideators, 6 adversarial verifiers, ~1.4M tokens) and synthesized/edited by the session orchestrator. Full verifier notes for corrected items are quoted inline; per-agent transcripts retained in the session workspace.